



# ***ADSP Debug Guide for OEMs***

**80-NA354-3 A**

**May 20, 2013**

**Submit technical questions at:**  
<https://support.cdmatech.com/>

## **Confidential and Proprietary – Qualcomm Technologies, Inc.**

**NO PUBLIC DISCLOSURE PERMITTED:** Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

**Restricted Distribution:** Not to be distributed to anyone who is not an employee of either Qualcomm or its subsidiaries without the express approval of Qualcomm's Configuration Management.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

Qualcomm reserves the right to make changes to the product(s) or information contained herein without notice. No liability is assumed for any damages arising directly or indirectly by their use or application. The information provided in this document is provided on an "as is" basis.

This document contains confidential and proprietary information and must be shredded when discarded.

Qualcomm is a trademark of QUALCOMM Incorporated, registered in the United States and other countries. All QUALCOMM Incorporated trademarks are used with permission. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer ("export") laws. Diversion contrary to U.S. and international law is strictly prohibited.

**Qualcomm Technologies, Inc.  
5775 Morehouse Drive  
San Diego, CA 92121  
U.S.A.**

**© 2013 Qualcomm Technologies, Inc.  
All rights reserved.**

# Contents

---

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>1 Introduction.....</b>                                                 | <b>6</b>  |
| 1.1 Purpose.....                                                           | 6         |
| 1.2 Scope.....                                                             | 6         |
| 1.3 Conventions .....                                                      | 6         |
| 1.4 References.....                                                        | 6         |
| 1.5 Technical assistance.....                                              | 7         |
| 1.6 Acronyms.....                                                          | 7         |
| <b>2 Live Debugging.....</b>                                               | <b>8</b>  |
| 2.1 Setting up the T32 environment.....                                    | 8         |
| 2.2 Using T32 for debugging.....                                           | 9         |
| 2.2.1 Attaching T32 and loading symbols for on-target debugging .....      | 9         |
| 2.2.2 Using T32 for debugging on MSM8974.....                              | 15        |
| 2.2.3 Interpreting T32 menu items.....                                     | 18        |
| 2.2.4 QuRT thread list .....                                               | 19        |
| 2.2.5 Understanding debug buffers and merged trace buffers .....           | 21        |
| 2.3 Determining if ADSP is power collapsed .....                           | 22        |
| 2.4 Measuring ADSP voltage .....                                           | 22        |
| <b>3 Crash Debugging.....</b>                                              | <b>23</b> |
| 3.1 Collecting RAM dumps using QPST.....                                   | 23        |
| 3.2 Dumping DSP memory including triggering NMI exception using T32.....   | 24        |
| 3.3 LPASS watchdog bite and timeout issues .....                           | 26        |
| 3.3.1 LPASS watchdog support.....                                          | 26        |
| 3.4 Using T32 for crash debugging.....                                     | 28        |
| 3.4.1 Finding the reference LPASS build .....                              | 28        |
| 3.4.2 Crashman tool for memory dump analysis .....                         | 29        |
| 3.4.3 Ensuring cache is flushed .....                                      | 31        |
| 3.4.4 QuRT system state variable .....                                     | 32        |
| 3.4.5 Types of exceptions .....                                            | 33        |
| 3.4.6 Determining if external NMI or self-NMI was received from dump ..... | 35        |
| 3.4.7 Example for debugging an exception .....                             | 36        |
| 3.4.8 On MSM8974 .....                                                     | 39        |
| 3.5 Debugging bootup issues .....                                          | 40        |
| 3.5.1 Checking boot-up process from RAM-dumps.....                         | 42        |
| 3.6 Checking for memory corruption.....                                    | 43        |
| 3.7 Stack frame analysis .....                                             | 44        |
| 3.7.1 Scripts to reconstruct callstacks.....                               | 45        |

|                                                                                                |           |
|------------------------------------------------------------------------------------------------|-----------|
| <b>4 Simulator Debugging .....</b>                                                             | <b>46</b> |
| <b>5 General Debugging .....</b>                                                               | <b>47</b> |
| 5.1 Hexagon log analysis .....                                                                 | 47        |
| 5.2 Hexagon messages for audio record .....                                                    | 47        |
| 5.2.1 Hexagon messages for Audio Playback.....                                                 | 53        |
| 5.2.2 Device switch sequence.....                                                              | 57        |
| 5.2.3 Fast forward/rewind sequence during playback .....                                       | 63        |
| 5.3 Ulog analysis.....                                                                         | 66        |
| 5.3.1 MPPM log .....                                                                           | 66        |
| 5.3.2 Sleep log .....                                                                          | 66        |
| 5.3.3 RPMMaster log.....                                                                       | 71        |
| 5.3.4 NPA log.....                                                                             | 72        |
| 5.4 Voice troubleshooting.....                                                                 | 74        |
| 5.4.1 Voice call is not working.....                                                           | 74        |
| 5.4.2 Voice is garbled during voice call .....                                                 | 74        |
| 5.4.3 Voice choppiness during voice call .....                                                 | 74        |
| 5.5 Audio troubleshooting .....                                                                | 74        |
| 5.5.1 Audio playback.....                                                                      | 75        |
| 5.5.2 Audio pre/postprocessing .....                                                           | 76        |
| 5.5.3 Audio recording.....                                                                     | 77        |
| <b>6 General Information .....</b>                                                             | <b>78</b> |
| 6.1 Understanding LPASS bootup, initialization, and sleep sequence on<br>MSM8960/APQ8064 ..... | 78        |
| 6.2 Finding the physical→virtual address mapping for ADSP image sections .....                 | 79        |

## Figures

|                                                                                                            |    |
|------------------------------------------------------------------------------------------------------------|----|
| Figure 3-1 System flow diagram for LPASS watchdog support.....                                             | 27 |
| Figure 6-1 Bootup, initialization, and power collapse timing diagram for LPASS on<br>MSM8960/APQ8064 ..... | 78 |

## Tables

|                                                  |    |
|--------------------------------------------------|----|
| Table 1-1 Reference documents and standards..... | 6  |
| Table 5-1 Sleep logs.....                        | 66 |

Qualcomm  
Confidential - May Contain Trade Secrets  
2026-05-12 06:30:01 GMT  
hyungchul.won@lge.com

## Revision history

| Revision | Date     | Description     |
|----------|----------|-----------------|
| A        | May 2012 | Initial release |

Qualcomm  
Confidential - May Contain Trade Secrets  
2026-05-12 06:30:01 GMT  
hyungchul.won@lge.com

# 1 Introduction

## 1.1 Purpose

This document presents methods for debugging the ADSP system on the MSM8660/MSM8260 and newer chipsets. It also provides setup and configuration requirements for tools used to debug the ADSP, such as Trace32 (T32).

## 1.2 Scope

This document is for Hexagon™ Access Program ISVs who have access to LPASS builds for MSM8660/MSM8260 and newer chipsets.

## 1.3 Conventions

Function declarations, function names, type declarations, and code samples appear in a different font, e.g., `#include`.

Code variables appear in angle brackets, e.g., `<number>`.

Commands to be entered appear in a different font, e.g., `copy a:*. * b:.`

Button and key names appear in bold font, e.g., click **Save** or press **Enter**.

## 1.4 References

Reference documents are listed in [Table 1-1](#). Reference documents that are no longer applicable are deleted from this table; therefore, reference numbers may not be sequential.

**Table 1-1 Reference documents and standards**

| Ref.                  | Document                                                                       |              |
|-----------------------|--------------------------------------------------------------------------------|--------------|
| Qualcomm Technologies |                                                                                |              |
| Q1                    | Application Note: Software Glossary for Customers                              | CL93-V3077-1 |
| Q2                    | Hexagon™ Multimedia: Tools Installation and Verification                       | 80-N3436-1   |
| Q3                    | Hexagon™ Multimedia: MSM8960 Master Document List                              | 80-N4639-1   |
| Q4                    | Hexagon™ Multimedia: Audio API Interface Specification                         | 80-N3007-1   |
| Q5                    | Hexagon™ Multimedia – Audio PCM/Bitstream Logging Through QXDM Professional    | 80-N3470-4   |
| Q6                    | Presentation: MSM8974 Snapdragon™ Sensors Core (SSC) Deep Dive                 | 80-NA157-92  |
| Q7                    | Application Note: Dedicated Sensors Processor Subsystem (DSPS) Debugging Tools | 80-N6957-1   |

## 1.5 Technical assistance

For assistance or clarification on information in this guide, submit a case to Qualcomm Technologies, Inc. (QTI) at <https://support.cdmatech.com/>.

If you do not have access to the CDMATech Support Service website, register for access or send email to [support.cdmatech@qualcomm.com](mailto:support.cdmatech@qualcomm.com).

## 1.6 Acronyms

For definitions of terms and abbreviations, see [Q1].

Qualcomm  
Confidential - May Contain Trade Secrets  
2026-05-12 06:30:01 GMT  
hyungchul.won@lge.com

## 2 Live Debugging

---

### 2.1 Setting up the T32 environment

**NOTE:** QTI is not responsible for distributing T32 installation or any updates associated with T32. Contact Lauterbach for any required assistance.

#### On MSM8660

In the following example, Hexagon™ Tools ver 1.2.08 is used; however, the present working tools version must be considered for the changes in this example. For this setup:

1. Install the Hexagon T32 update by copying the files from the latest update directory on top of the T32 base installation. (This may replace already existing files.)
2. The Hexagon MCD library is installed as part of the Hexagon Tools Release package and is located at C:\Qualcomm\HEXAGON\_Tools\1.2.08\qc\lib\iss\T32\_MCD.dll.

The directory containing T32\_MCD.dll must be included in your path environment variable; or this file can be copied to the C:\T32 directory. For more information on Hexagon Tools installation, see [Q2].

3. In DOS, enter the following command to create a folder c:\t32work with full permissions.

```
chmod 777 c:\t32work
```

This directory stores temporary T32 files.

#### On MSM8960/APQ8064

Follow the same steps as in the example above; however, use Hexagon Tools ver 3.0.10 instead.

#### On MSM8974/APQ8074

Follow the same steps as in the example for MSM8660; however, use Hexagon Tools ver 5.0.10 or later. Also update the T32 directory with the latest patch from Lauterbach that also supports HexagonV5 binaries.

**NOTE:** Use Dec 2012 version with the appropriate patch.



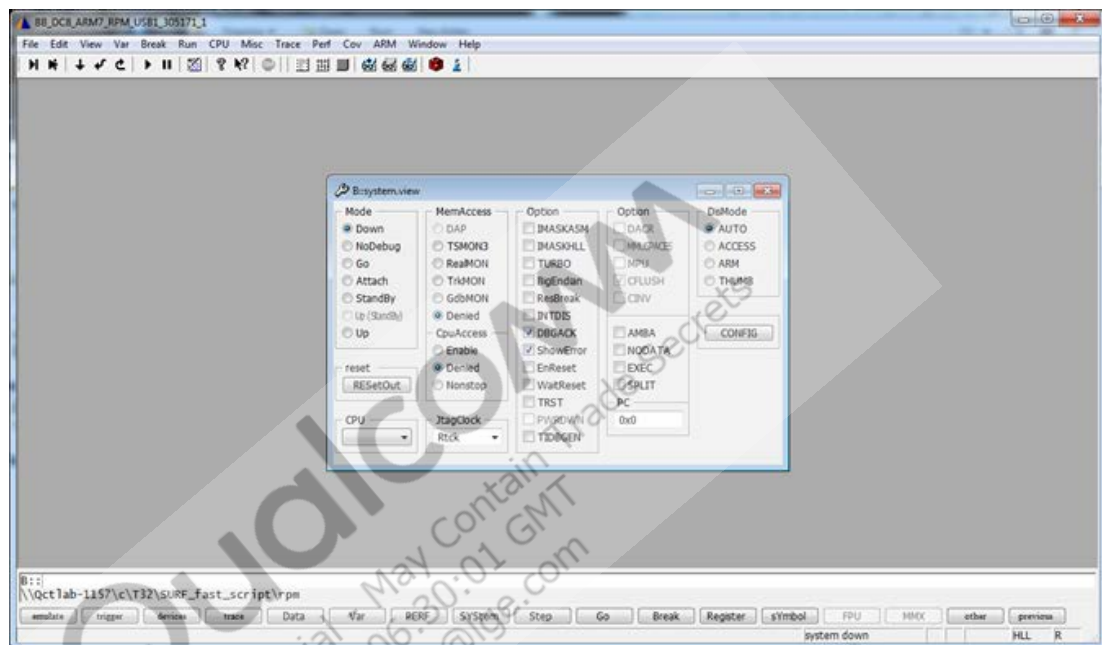
## 2.2 Using T32 for debugging

### 2.2.1 Attaching T32 and loading symbols for on-target debugging

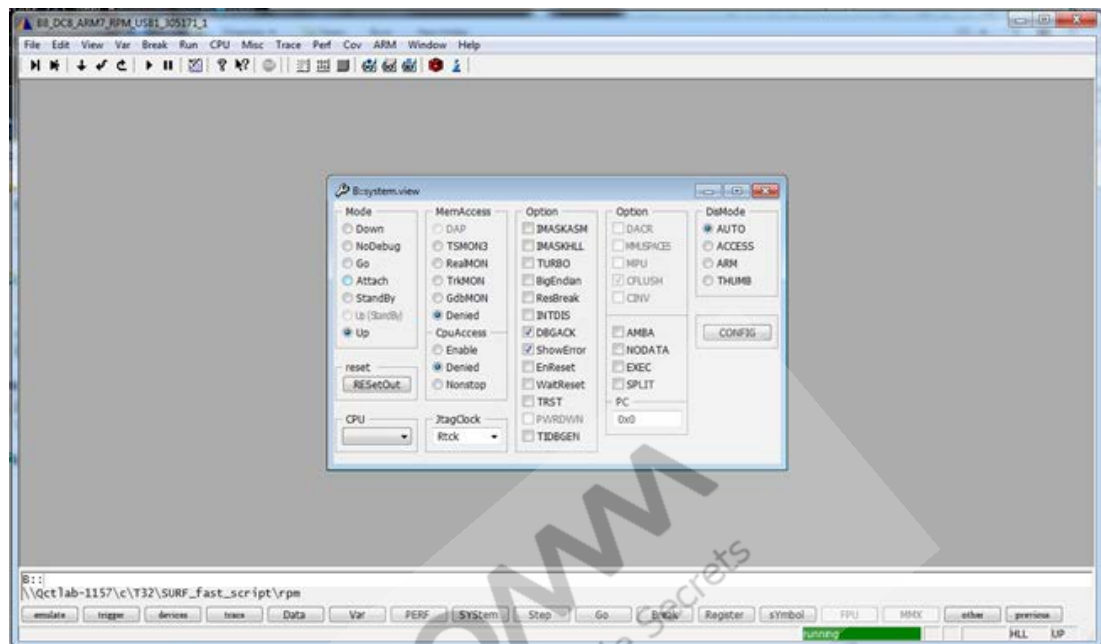
Assuming that the target is running, and after connecting the JTAG cable between the PC and the target, a debug session can be started. The T32 files and shortcuts referenced in this section are:

- On the MSM8660, from <Modem build root>\modem\_proc\tools\t32\t32\_dynamic\
- On the MSM8960/APQ8064, from <Meta build root>\common\t32\t32\_dynamic\

1. Double-click the **Dynamic\_ARM7\_RPM** shortcut and a window will open.



2. Once the main user interface screen displays on-target, click **Attach** in the Mode section of System.view, or type system.mode.attach at the Dynamic\_ARM7\_RPM T32 prompt. The transition of the RPM T32 window from system-down to running appears.

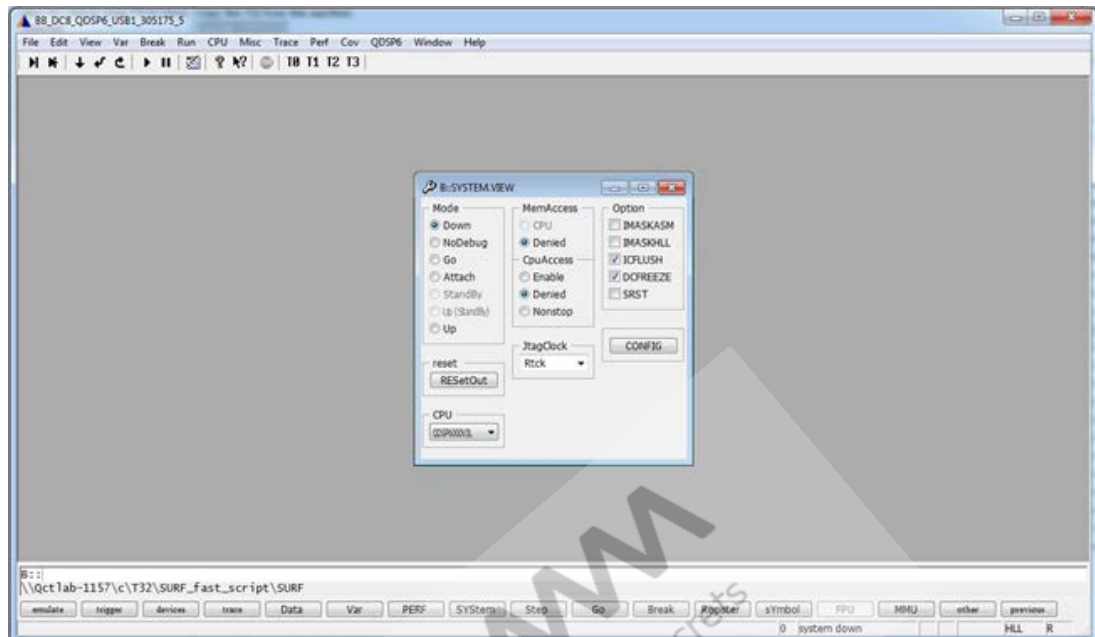


**NOTE:** In the system.view window, although **Up** is selected in the Mode section, only click **Attach**.

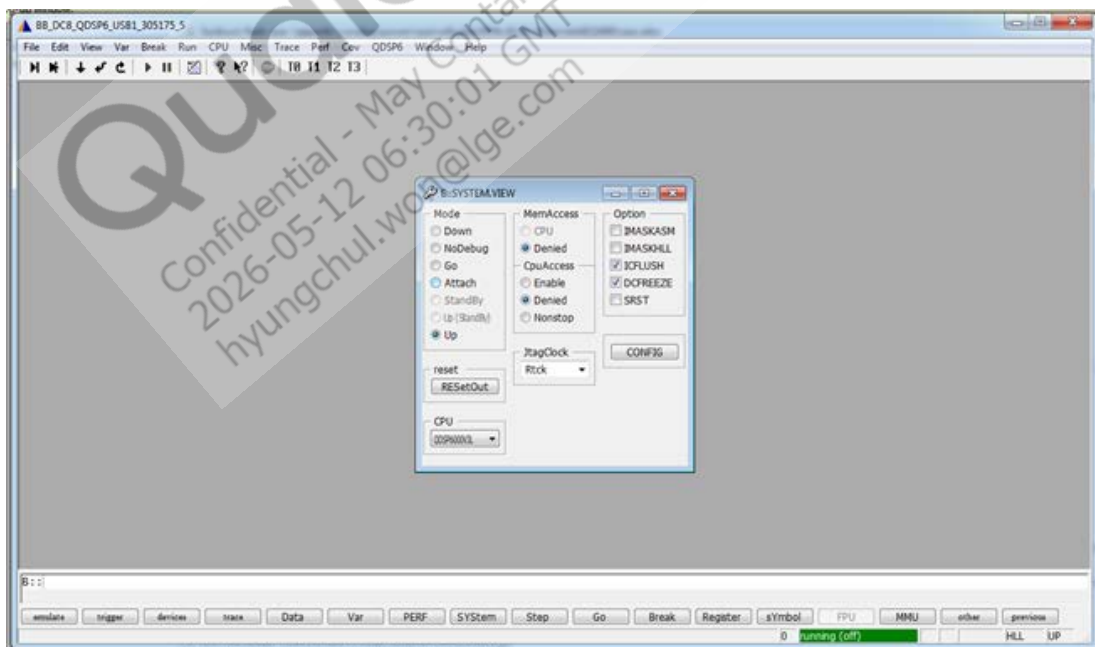
Steps 1 and 2 get the ARM7™ RPM up and running.

**NOTE:** The Hexagon processor and T32 cannot be attached if there is no power to the Hexagon processor. An easy way to pull the Hexagon out of power-collapse is to play audio or make a voice call.

3. Double-click the **Dynamic\_QDSP6\_ADSP** shortcut, and the Dynamic\_QDSP6\_ADSP window opens.



4. Click **Attach** in the Mode section of System.view, or type system.mode.attach at the Dynamic\_QDSP6\_ADSP T32 prompt. This window is where a programmer can debug the Hexagon code; this window with transition from system-down to running is shown here.



**NOTE:** In the system.view window, although **Up** is selected in the Mode section, only click **Attach**.

- 1           5. In the DC8\_QDSP6 window, CD to the reference build with which the target is loaded, e.g.,

2  
3           `cd C:\M8660HAPINT602215\`

- 4  
5           6. Break the Hexagon code by clicking **Break** in the T32 menu (the Break button is the  
6           pause/parallel button next to run  **break**).

7       **NOTE:** Keep the volume low on-target before breaking the Hexagon code. Once the Hexagon code is in  
8       Break mode, the Hexagon processor keeps sending its last buffer of data through the speakers,  
9       which could result in any periodic noise-like tones.

- 10          7. Run the following command to load symbols and set environment variables for debugging;

11  
12          `do qdsp6/scripts/DSP_Connect.cmm <dsp_start_address>`

13  
14          On the MSM8660, e.g., run the following command:

15  
16          `do qdsp6/scripts/DSP_Connect.cmm 0x46700000`

17  
18          On the MSM8960/APQ8064, run the following command:

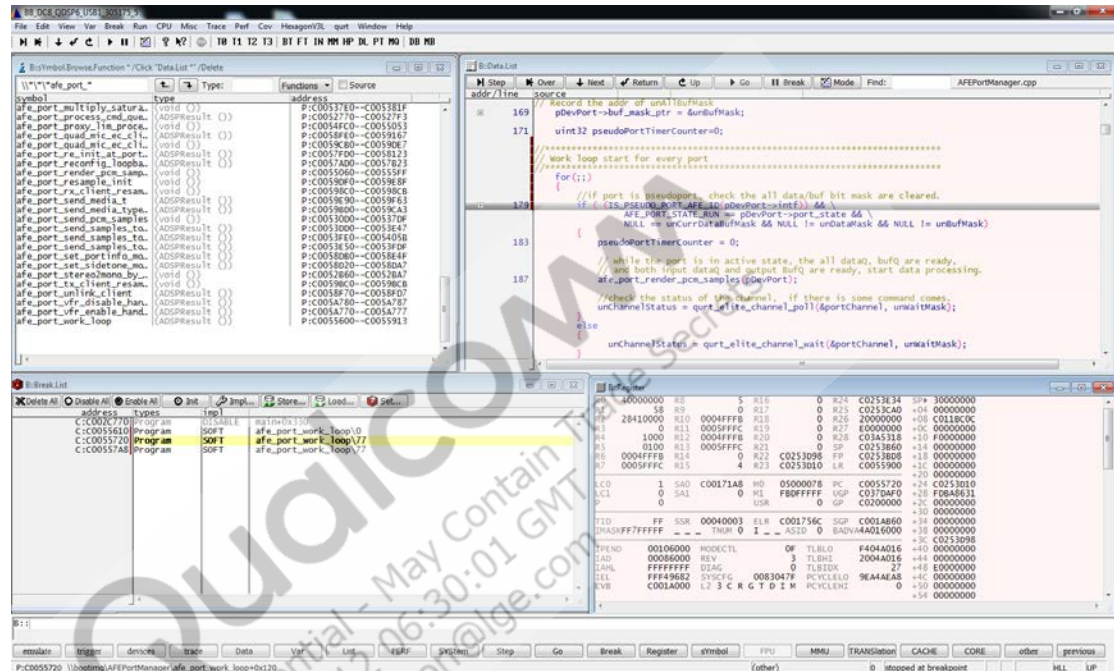
19  
20          `do qdsp6/scripts/DSP_Connect.cmm 0x8da00000`

- 21  
22          8. New breakpoints can be set, code can be step-executed, and other general debug options can  
23          be performed. To view the breakpoints, click **Break – List**. Double-clicking any listed  
24          breakpoint opens a new source window associated with this breakpoint.  
25          9. If the breakpoint location/function is known, one efficient way to set new breakpoints is to  
26          select **Break – Set**.

27          Another option is to select **Var – Show Functions**, and type the name of the function in the  
28          Symbol.Browse.Function window (wildcards such as \* can also be used for an efficient  
29          search). Double-click the function name to open the source file of the function, and  
30          double-click any line to set a new breakpoint. The updated breakpoints list can be seen by  
31          clicking **Break – List**.

The following sub-windows can be displayed during live ADSP debugging through:

- Symbol.Browse.Function window appears on top left. A search for functions `afe_port_*` shows all the functions with names `afe_port_`.
- Double-click **afe\_port\_work\_loop** to display the Data.list `afe_port_work_loop` window on the top right. A new breakpoint on line 179 of `afe_port_work_loop` is set by double-clicking this line (shown with broken red lines).
- Break.list window appears on the lower left with an updated breakpoint at `afe_port_work_loop/77`.
- Register window appears on the lower right.



**NOTE:** If a breakpoint is not set at the expected line in C/C++ code, it could be the result of code optimization. Hexagon tools provide various code optimization options for efficient usage of the Hexagon hardware that are given at compile time. They are `-O0` (no optimization), `-O1`, `-O2`, `-O3`, and `-Os` (space/memory optimization). When an optimization level (except `-O0`) is used, the Hexagon assembly instructions generated from C/C++ code may be grouped into packets with other assembly instructions, and their location may also be reordered. This can cause breakpoints to not be set at the expected line in C/C++ code, and it can also cause stepping through of C/C++ code to be nonsequential.

10. Press **run/go** to hit the breakpoint. To view the source, click **View – List Source**.

```

169 // Record the addr of unwaitMask;
170 pDevPort->buf_mask_ptr = &unwaitMask;
171 uint32 pseudoPortTimerCounter=0;
172
173 //work loop start for every port
174 for(;;)
175 {
176     //if port is pseudoport, check the all data/buf bit mask are cleared.
177     if (IS_PSEUDO_PORT_AFE_ID(pDevPort->intf) && \
178         AFE_PORT_STATE_RUN == pDevPort->port_state && \
179         NULL == unCurrDataBufMask && NULL != unDataMask && NULL != unBufMask)
180     {
181         pseudoPortTimerCounter = 0;
182         // while the port is in active state, the all data, bufq are ready,
183         // and both input dataq and output bufq are ready, start data processing.
184         afe_port_render_pcm_samples(pDevPort);
185         //check the status of the channel, if there is some command comes.
186         unChannelStatus = qurt_elite_channel_poll(&portChannel, unwaitMask);
187     }
188     else
189     {
190         unChannelStatus = qurt_elite_channel_wait(&portChannel, unwaitMask);
191     }
192     if (unChannelStatus & qurt_elite_queue_get_channel_bit(&pDevPort->cmdq))
193     {
194         // call common utility to process command queue in standard way.
195         result = afe_port_process_cmd_queue(pDevPort, &pDevPort, afe_port_cmd_handler_ptr);
196     }
197     // If service has been destroyed, exit.
198     if (ADSP_ETTERMINATED == result) return ADSP_EOK;
199     if (ADSP_FAILED(result))
200     {
201         MSG_L(MSG_SSD0_QDSP6, DBG_ERROR_PRI0, " message FAIL Ndx", result);
202     }
203     else
204     {
205         MSG_L(MSG_SSD0_QDSP6, DBG_HIGH_PRI0, " message SUCCESS Ndx", NULL);
206     }
207     if (unChannelStatus & qurt_elite_signal_get_channel_bit(pDevPort->signal))
208     {
209         qurt_elite_signal_clear(pDevPort->signal);
210         if (AFE_PORT_STATE_RUN == pDevPort->port_state && \
211             AFE_PORT_TIMING_MODE_FTRT != pDevPort->timing_mode)
212         {
213             if (pDevPort->dir == AFE_PORT_RX_DIR) //hit for pseudo port in RT mode, since pseudo port is RX DIR always
214             {
215                 if (IS_PSEUDO_PORT_AFE_ID(pDevPort->intf) && (NULL != unDataMask || NULL != unBufMask))
216                 {
217                     pseudoPortTimerCounter++; //counts up
218                     if (pseudoPortTimerCounter > AFE_PSEUDO_PORT_TIMER_MODE_THRESH)
219                     {
220                         //only timer mode enters here.
221                         afe_port_render_pcm_samples(pDevPort); //sends zeros if rx buffers not available.
222                         pseudoPortTimerCounter = AFE_PSEUDO_PORT_TIMER_MODE_THRESH; //on that when command is ok
223                     }
224                 }
225             }
226         }
227     }
228 }

```

The line highlighted in gray is the present location where the execution is halted. Step-in, step-over, run/go, and other debug buttons from the T32 menu can be used to debug the code.

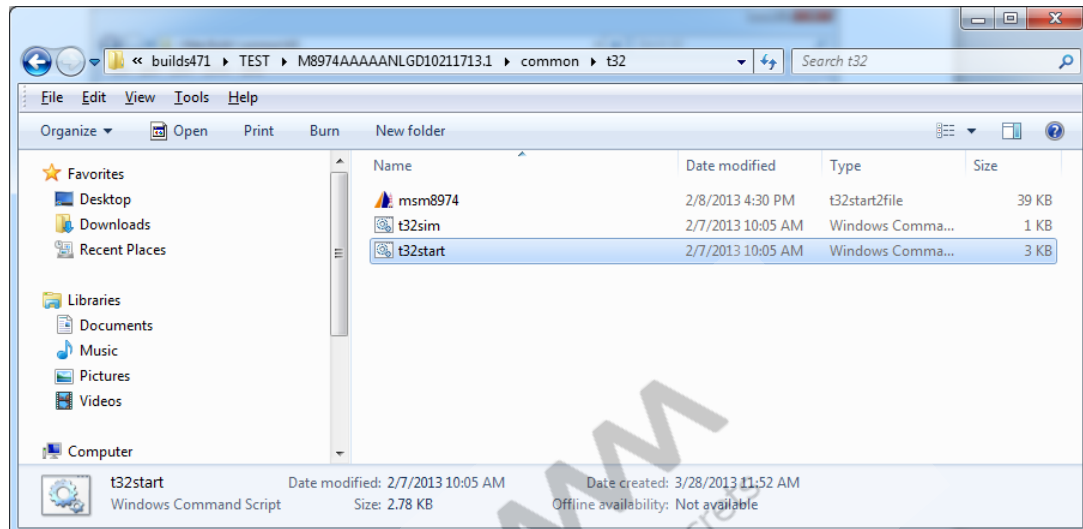
11. If for some reason T32 is slow/unresponsive/hangs after attaching ADSP for on-target debug, disable RPM LPM with the following ADB command

```
echo 0 > /sys/module/rpm_resources/enable_low_power/rpm_cpu
```

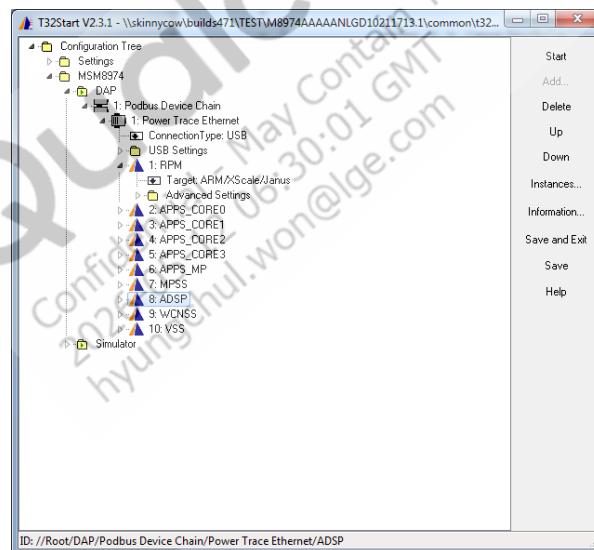


## 2.2.2 Using T32 for debugging on MSM8974

1. Double-click **t32start.cmd** script to start msm8974 t32starter2file (.ts2) and expand to 'Configuration Tree – MSM8974 – DAP – Podbus Device Chain.'



- To have on-target debug, the T32 windows 'RPM', 'APPS\_CORE0' and 'ADSP' need to be opened.



2. Right-click **RPM**, then click **start**, or left-click **RPM** and then press **Start** in the right panel.
3. Attach RPM by issuing sys.m.a command at B:: prompt.
4. Open APPS\_CORE0, then type sys.m.a at B:: prompt to attach RPM and APPS\_CORE0.
5. Play an audio file on-target to bring ADSP active (out of power collapse).
6. Open ADSP T32 window and issue sys.m.a command to attach to ADSP.

7. At the B: prompt, cd to the adsp\_proc directory that was loaded on-target,  
cd <ADSP Image> \adsp\_proc. Break ADSP and APPS\_CORE0 windows. When the audio  
stops playing, issue the following commands

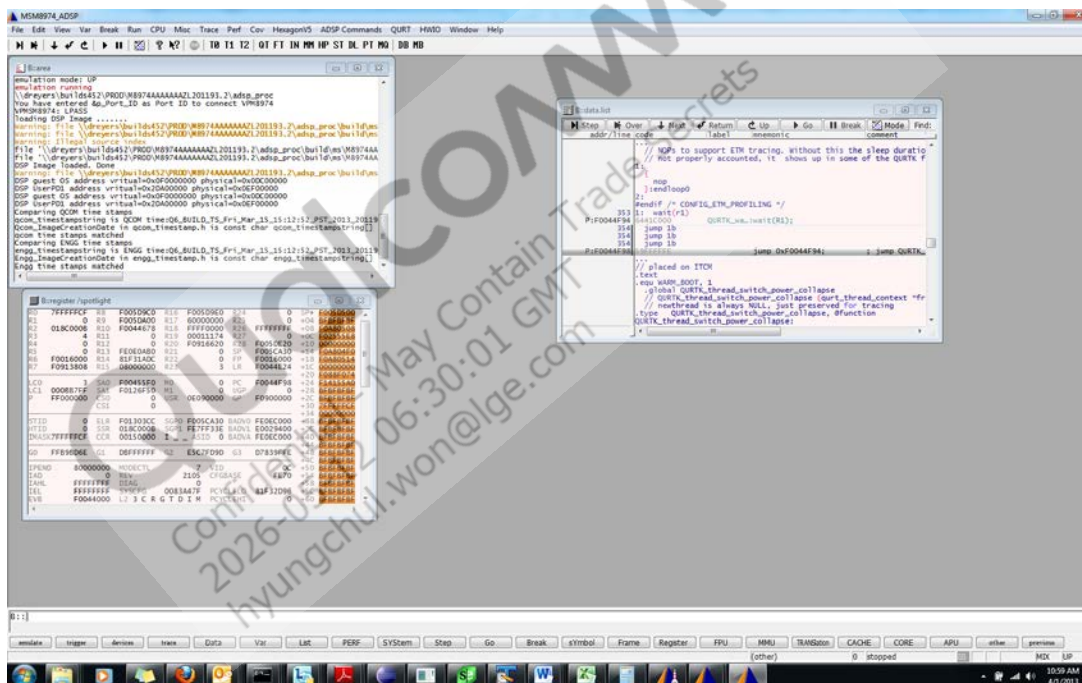
- Single-Protection Domain (SPD) builds:

```
do qdsp6\scripts\msm8974\DSP_connect_load_symbols.cmm
do qdsp6\scripts\msm8974\DSP_load_osam.cmm
```

- Multiprotection Domain (MPD) builds:

```
do qdsp6\scripts\msm8974\DSP_connect_load_symbols_MPD.cmm
do qdsp6\scripts\msm8974\DSP_load_osam_MPD.cmm
```

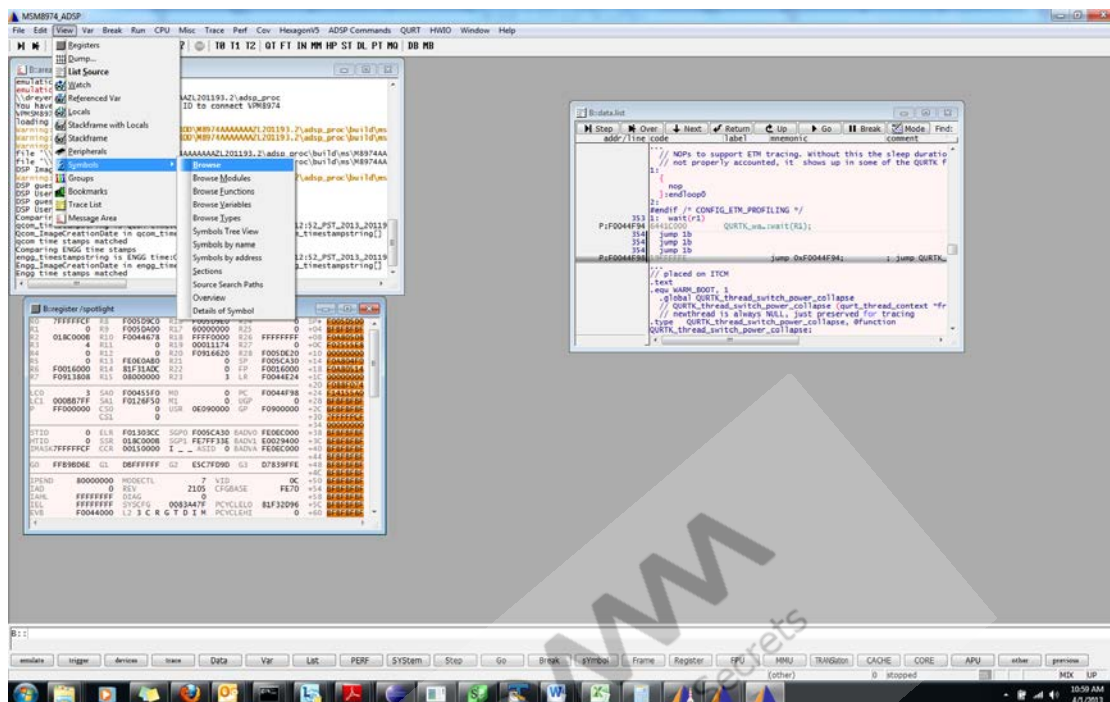
8. Set breakpoints in the code where intended, then issue play in the APPS\_CORE0 and ADSP  
windows to play the audio file on target and see code halting after hitting these break points.





1

One can search for symbols at **View – Symbols – Browse**.

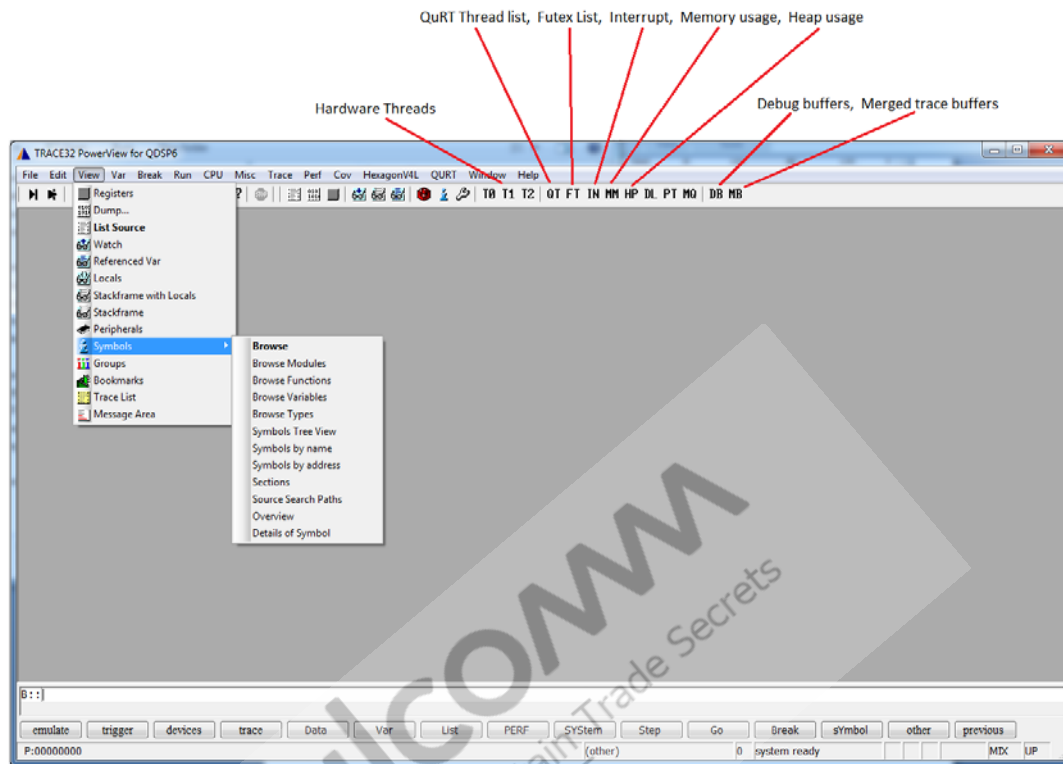


2

3

## 2.2.3 Interpreting T32 menu items

The T32 main menu for ADSP debugging is shown here.



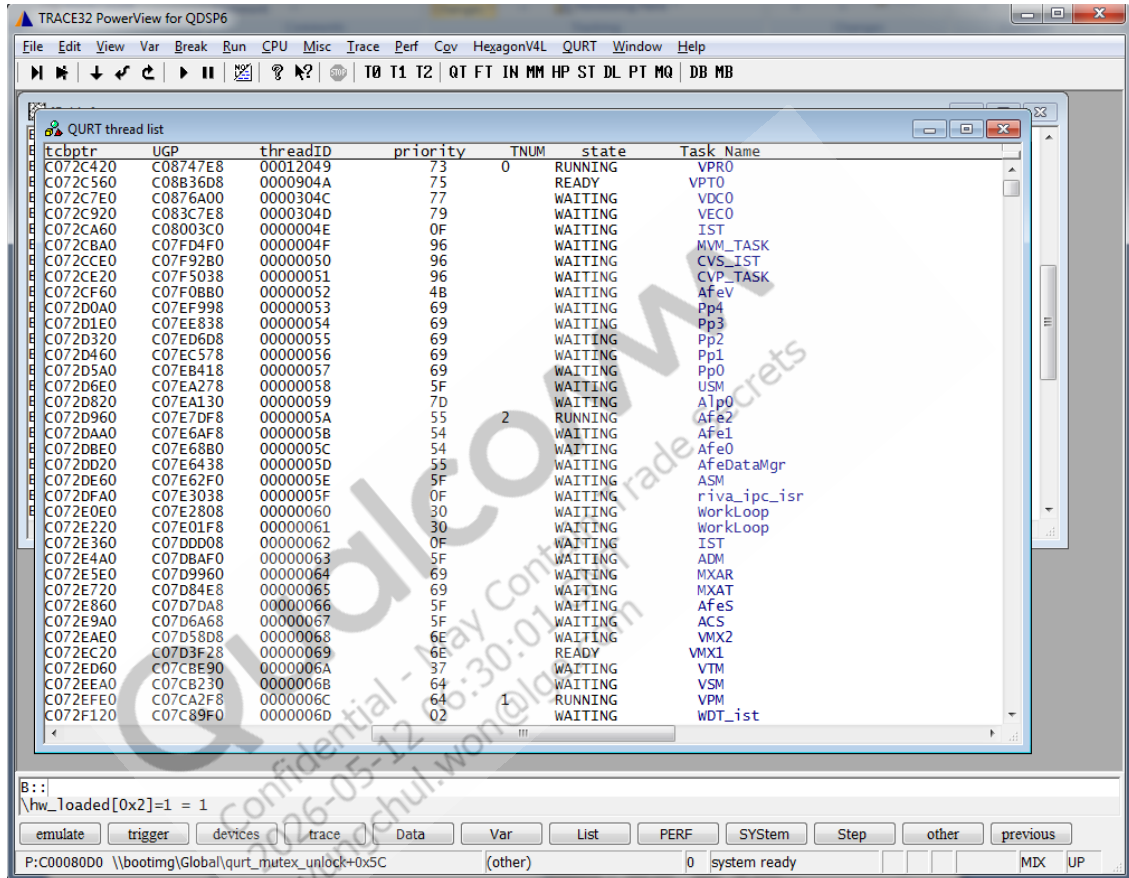
1. Type **r** at B:: to display registers.
  - T0, T1, and T2 are the hardware threads for MSM8960/APQ8064. On the MSM8660, there are four hardware threads; T0 to T3.
2. Click the **T0** button to display the hardware registers for thread T0. (Displaying registers is similar for the other threads.)
  - QT is the QuRT™ threads button and displays the QuRT software threads at the time the dump was taken. See Section 2.2.4 for more information.
  - The **FT** button is the futex list and displays all the engaged futexes and displays any deadlocks if occurred.
  - **IN** displays the interrupt list.
  - **MM** lists the free memory blocks available.
  - **HP** is the heap usage while the dump was being extracted.
  - **DB** shows the debug buffer. The printf messages are stored in the debug buffer. More information regarding this buffer is provided in Section 2.2.5.
  - **MB** is the merged trace buffers. Trace buffers provide an association of which QuRT software thread is running on a particular hardware thread and also context switches; MB display merges all the hardware threads. See Section 2.2.5 for more information.

To view a symbol and its value, click **View→Symbols→Browse** and search for the symbol in the search box. You can also browse for functions, modules, variables, etc.

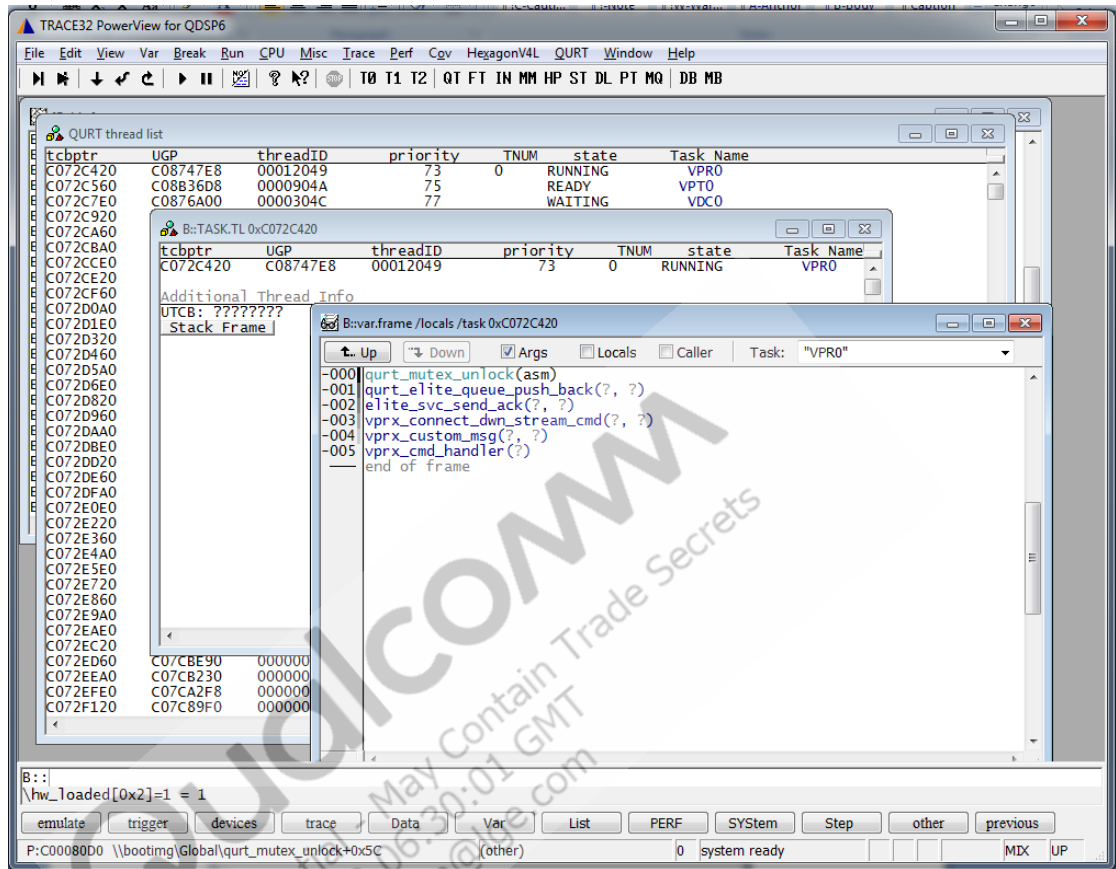
## 2.2.4 QuRT thread list

This list displays all created software threads, and includes information regarding task name, thread priority, and the hardware thread on which the software thread is running (if applicable), e.g.:

- Thread VPR0 is running on hardware thread 0.
- VPM is running on hardware thread 1.
- Afe2 is running on hardware thread 2.

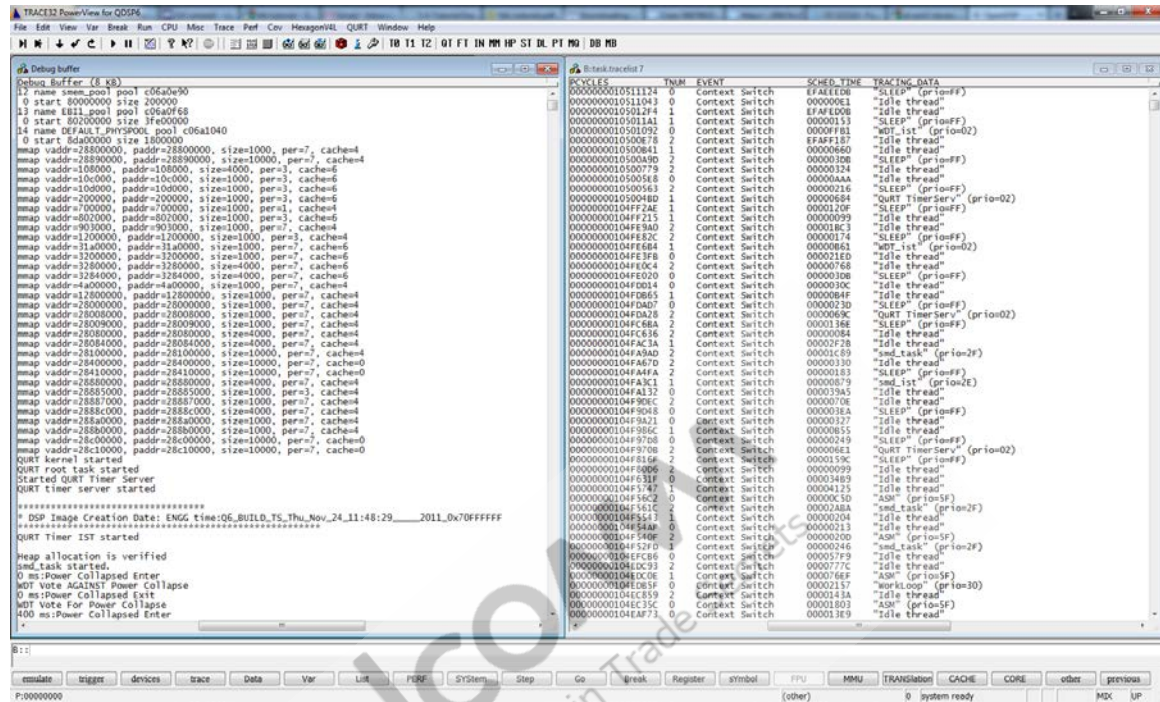


1. Click the **tcbptr** value for a given thread to display some additional thread information.
2. Click the **Stack Frame** button to display the call stack for the thread. The following screenshot shows that function `qurt_mutex_unlock` is at the top of the call stack for thread `VPR0`.



## 2.2.5 Understanding debug buffers and merged trace buffers

Debug buffers and merged trace buffers displayed in T32 are shown here.



The window on the left is the debug buffers, displayed by clicking the **DB** button. This window shows printf messages. The main messages displayed are heap initialization, QuRT initializations, power collapse enter and exit, and time of power-collapse enter and exit.

The window on the right is the merged trace buffers, displayed by clicking the **MB** button. QuRT maintains trace buffers for each hardware thread, which can be viewed by clicking **QuRT-Trace Buffers** from the main menu items. MB merges the buffers associated with each hardware thread into one and displays them.

PCYCLES is the processor cycle count when the task in TRACING DATA (displays the priority of the task) started running or resumed on the hardware thread TNUM.

SCHED\_TIME is the number of consecutive processor cycles on that TNUM for the given task.

Looking at the next two tasks on TNUM 0, Idle Thread has schedule time E1 with PCYCLES 10511043, and WDT\_ist has schedule time FFB1 with PCYCLES 10501092, thus subtracting both the processor cycles  $10511043 - 10501092 = \text{FFB1}$ , which is the SCHED TIME on task with WDT\_ist.



## 2.3 Determining if ADSP is power collapsed

The register QDSP6SS\_SLPC\_STATUS (register address is 0x28887020) shows whether the Hexagon processor is in power-collapse or not.

**0x7020 QDSP6SS\_SLPC\_STATUS**

Type: Read  
Clock: AHB\_CLK  
Reset State: NA

Sleep controller status register. This register is intended for debugging purposes.

**QDSP6SS\_SLPC\_STATUS**

| Bits | Name              | Description                        |
|------|-------------------|------------------------------------|
| 31:3 | RESERVED_BITS31_3 | reserved space.                    |
| 2:0  | FSM_STATE         | This is the SLPC FSM current state |

- If the value of QDSP6SS\_SLPC\_STATUS is equal to 0, this indicates that ADSP is not in power-collapse.
- For MSM8660, if this register is equal to 5, this indicates that the ADSP is in power-collapse.
- For MSM8960/APQ8064, if this register is equal to 3, this indicates that the ADSP is in power collapse.

## 2.4 Measuring ADSP voltage

To ensure ADSP voltage is at the expected level, measure the ADSP power rail with an oscilloscope or voltmeter. For more information regarding expected voltage levels for the MSM8660 and MSM8960/APQ8064, see Section 3.6.

- For MSM8660, the ADSP power rail is VREG\_S3B.
- For MSM8960/APQ8064, the ADSP power rail is VREG\_L26.

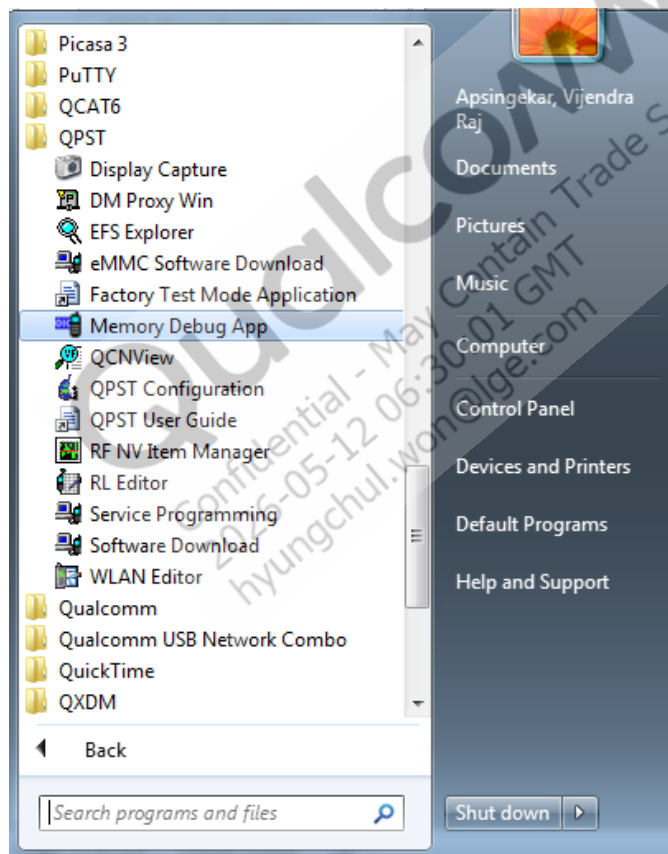
## 3 Crash Debugging

There are two ways to get RAM dumps after a crash:

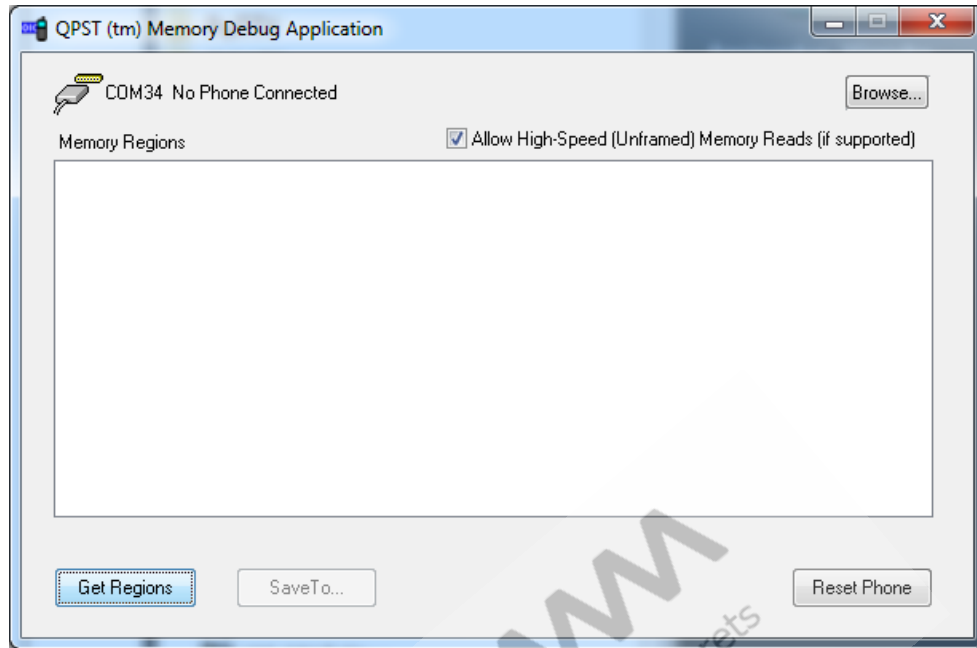
- Using the QPST Memory Debug app
- Using T32

### 3.1 Collecting RAM dumps using QPST

1. Once the phone goes into Download mode, connect the USB and open the Memory Debug app as part of the QPST installations.



2. Click **Get Regions** to collect the memory dumps and save them using the **Browse** button.



## 3.2 Dumping DSP memory including triggering NMI exception using T32

ADSP memory is part of EBI RAM. Get the ADSP dump through the T32 environment as follows.

### For MSM8660

1. In the modem build, change the directory to \modem\_proc\tools\t32\t32\_dynamic.
2. Launch Dynamic\_ARM7\_RPM and attach.
3. Launch Dynamic\_ARMv7\_SCORPION0 and attach.
4. Break in Dynamic\_ARMv7\_SCORPION0 window.
5. Set the ADSP NMI bit to save ADSP registers and flush the ADSP cache by entering the following command:

```
PER.S AZSD:0x28800010 %LONG 1
```

6. Save the DSP dump:

```
d.save.binary c:\temp\dump.bin az:0x46700000++0x1800000
```



**For MSM8960/APQ8064**

1. Open RPM T32.
2. Attach RPM T32.
3. Open Krait0 T32.
4. Break Krait0 T32.
5. Set the ADSP NMI bit on Krait0 T32:

```
PER.S AZSD:0x28800010 %LONG 1
```

6. Save the ADSP dump on Krait0 T32:

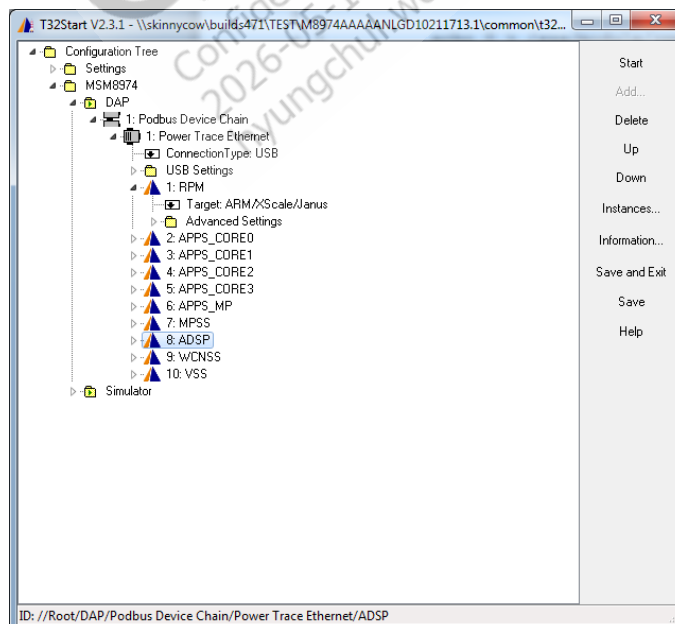
```
d.save.binary c:\temp\dump.bin az:0x8da00000++0x1800000
```

**For MSM8974**

1. Open RPM T32.
2. Attach RPM T32.
3. Set the ADSP NMI bit:

```
PER.S EZAHB:0xfe200040 %LONG 1 //Assert the NMI
```

4. Save the ADSP dump: use d.save.binary <dump\_path>\dump\_name.bin EZAHB:0x0dc00000++0x1800000 for ADSP dumps.
5. Save the shared memory: use d.save.binary <shared\_mem\_path>\dump\_name.bin EZAHB:0xfa00000++0x1fffff for shared memory dumps.



## 3.3 LPASS watchdog bite and timeout issues

A detailed description of the LPASS watchdog feature is described in Section 3.4.1.

If an LPASS watchdog bite (timeout) occurs, the message watchdog bite received from [Q6] is printed in the Linux kernel logs. Typical reasons for an LPASS watchdog bite are:

- ADSP exception
- ADSP voltage or clock are OFF

If LPASS watchdog hardware registers are corrupted or a deadlock occurs in the ADSP, a watchdog bite may not occur. In this case, the apps or modem processors may try to send commands to the ADSP through APR, but the ADSP will be unresponsive. When this occurs, Linux kernel log messages are printed such as:

- <3>[ 448. 332085] apr\_tal: open timeout.
- <3>[ 448. 332212] APR: Unable to open handle.

### 3.3.1 LPASS watchdog support

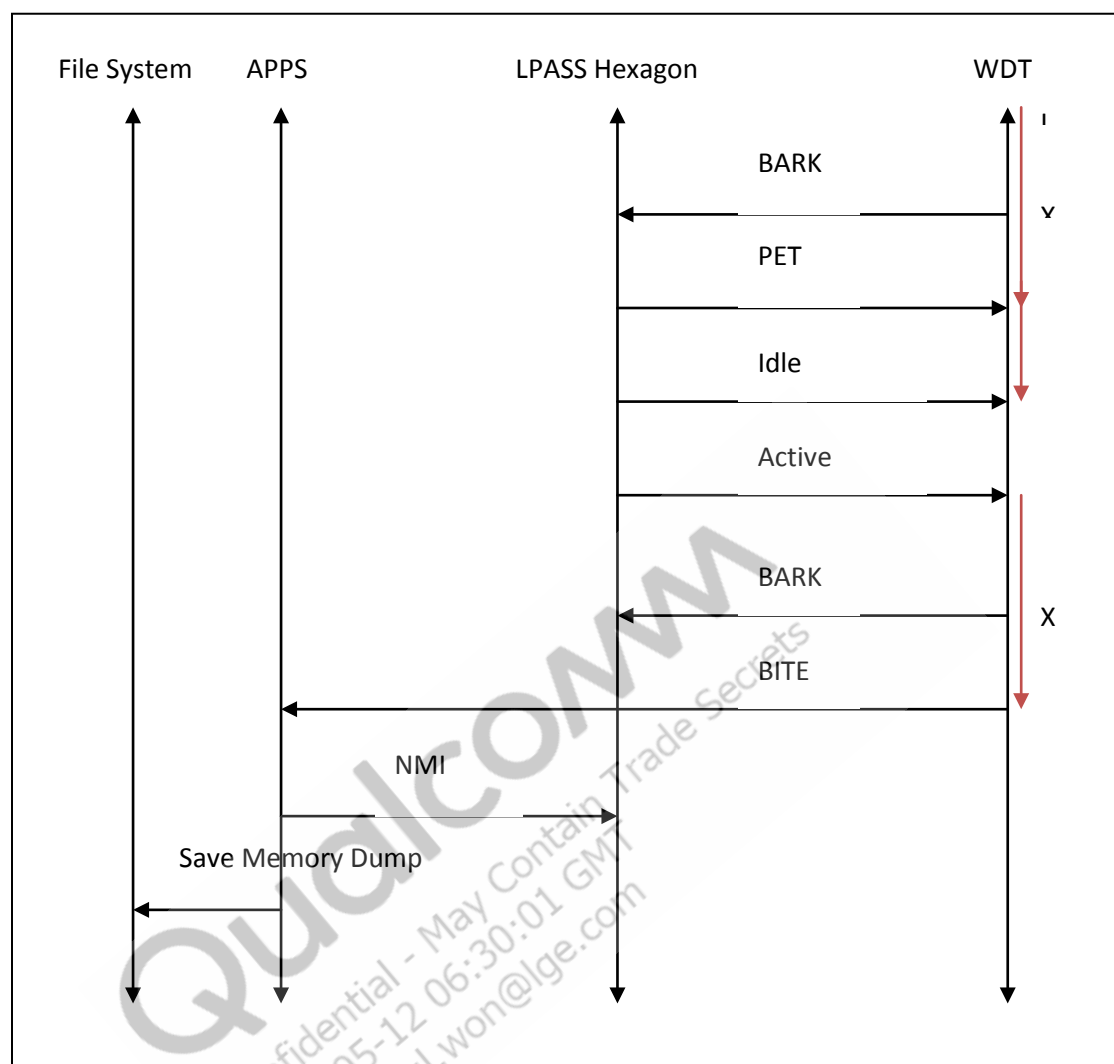
LPASS watchdog support includes:

- Support for a primary hardware watchdog event exists for the Hexagon processor, which generates an interrupt to another managing processor if it expires.
- This primary watchdog (bite) event triggers at a fixed duration of 390 ms measured from the time the watchdog timer was last reset.
- A secondary (bark) hardware watchdog event exists for the Hexagon processor that generates a high priority interrupt to Hexagon when it expires. This occurs at a fixed duration of 200 ms measured from the time the watchdog timer was last reset.

#### 3.3.1.1 Watchdog timer requirements

- The timer will be initialized at the beginning of the system.
- The timer will run for X time (200 ms) before it interrupts Hexagon. This is called bark time.
- The Hexagon processor must receive the interrupt and restart the timer. This is called pet the dog.
- If the timer was not reset (pet the dog) and continues to run to reach Y time (390 ms), the watchdog timer will interrupt the apps processor. This is called bite time.
- The timer will be disabled before power collapse.
- The timer will be re-enabled at powerup.

Figure 3-1 shows the system flow for LPASS watchdog support.



### Figure 3-1 System flow diagram for LPASS watchdog support

## 3.4 Using T32 for crash debugging

### 3.4.1 Finding the reference LPASS build

A timestamp is embedded in the Hexagon image that indicates the time the image was built. An example timestamp is Q6\_BUILD\_TS\_Thu\_Oct\_13\_12:07:12\_PDT\_2011\_0x10200500.

The Hexagon processor memory dump also has a timestamp, and the timestamp from the Hexagon image should match the timestamp from the Hexagon memory dump so that symbols match and the dump can be properly analyzed. All Hexagon memory dumps must contain the text Q6\_BUILD.

A fast way to determine if this text is present is to run the following in DOS:

```
strings <Hexagon memory dump> | grep Q6_BUILD
```

Where:

- <Hexagon memory dump> is the received Hexagon memory dump
- strings is a program that is installed with Cygwin in the binutils package

The Build ID in the above timestamp is 1200500.

#### On MSM8660

It is the customer's responsibility to have association of the known builds from QTI and the builds that customers built.

#### On MSM8960/APQ8064/MSM8974

Two timestamps are embedded, one the QTI original timestamp (QCOM), and the other is an engineering timestamp (ENGG):

```
QCOM time:Q6_BUILD_TS_Thu_Oct_13_12:07:12_PDT_2011_0x10200500
ENGG time:Q6_BUILD_TS_Sat_Oct_22_16:19:36_____2011_0x70FFFFFF
```

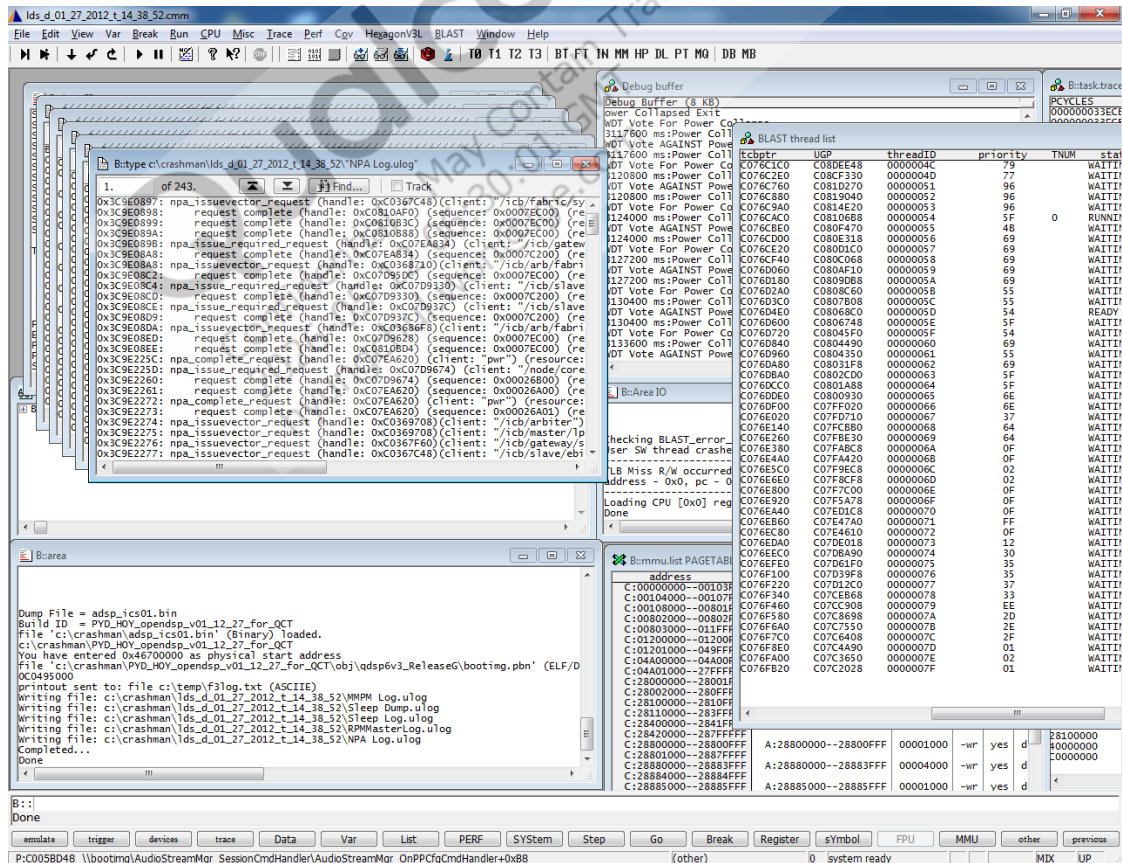
The timestamp from the dump should match ENGG time for debug purposes.

**NOTE:** On MSM8960/APQ8064, 70FFFFFF is a default engineering build ID; oOn MSM8974, the default engineering build ID is 0x8FFFFFFF. QCOM time never changes, however, ENGG time is updated every time a build is initiated.

### 3.4.2 Crashman tool for memory dump analysis

This section describes how to run the Crashman tool, given a memory dump and reference LPASS build. This tool loads the memory dump and symbols in T32, and then runs scripts to extract and display information for debugging. These scripts include:

- Software thread list – This is described in Section 2.2.4.
- Structure QuRT\_error\_info – If an exception occurred, information regarding the first exception that was encountered is printed, along with the software thread, instruction, and address that caused the exception (if applicable).
- Debug buffer – This is described in Section 2.2.5.
- F3 logs – These are the last Hexagon messages, stored in a 128 kB buffer, and are described in Section 5.1.
- Ulogs (MMPM Log, Sleep Dump, Sleep Log, RPMMasterLog, and NPA Log) – These are described in Section 5.3.
- Merged trace buffers – These are described in Section 2.2.5.
- QuRT interrupt list.
- QuRT memory usage – This shows the memory allocated by QuRT.
- Pagetable – This shows virtual-to-physical address mappings, permissions, and cache attributes specified by the MMU.



To run the Crashman tool:

1. Make sure you have T32 for Hexagon and that it is located at C:\T32.
2. Open a DOS prompt.
3. In the ADSP build, change directory to \qdsp6\scripts\crashman.

## For MSM8660

Run the loader batch file loadaddspdump.bat or loadmemdump.bat.

- For DSP memory dump only, which starts at address 0x46700000, run the following command:

```
loadaddspdump <name of the memory dump> <ADSP build root>
```

e.g., loadaddspdump.bat c:\temp\adsp\_dump.bin c:\temp\ADSP\_build

- For full EBI memory dump which starts at address 0x40000000, run the following command:

```
loadmemdump <name of the memory dump> <ADSP build root>
```

e.g., loadmemdump.bat c:\temp\EBICS0.BIN c:\temp\ADSP\_build

## For MSM8960/APQ8064

Run the loader batch file lpasscrashman.bat.

- For full EBI memory dump which starts at address 0x80000000, run the following command:

```
lpasscrashman.bat <TARGET> <name of the memory dump> <Directory to store logs> <ADSP build root>
```

e.g., lpasscrashman.bat 8960 c:\temp\EBICS0.BIN c:\temp\crashman\_logs

c:\temp\ADSP\_build or

lpasscrashman.bat 8064 c:\temp\EBICS0.BIN c:\temp\crashman\_logs c:\temp\ADSP\_build

**NOTE:** The <ADSP build root> provided should be one level above the lpass\_proc directory.

T32 should start up with the current memory dump loaded and analyzed.

## For MSM8974

1. Get the latest version of crashman tools from

<Qualcomm ADSP release>\adsp\_proc\qdsp6\scripts\

2. For Full EBI memory dump , use the following

```
adspcrashman.bat 8974 <memory dump file> <log file location> <Customer  
build loaction>
```

e.g., adspcrashman.bat 8974 DDRCS0.BIN c:\temp\logs c:\8974\_adsp\_build

**NOTE:** The <ADSP build root> provided should be one level above the adsp\_proc directory.

### 3.4.3 Ensuring cache is flushed

To properly analyze an ADSP memory dump, the ADSP cache must first be flushed (otherwise some memory contents may not be updated as expected). The ADSP cache is flushed if an exception occurs in the ADSP, or if the ADSP goes to power-collapse. As described in Section 3.1, before dumping ADSP memory, the NMI bit must first be set to trigger a Nonmaskable Interrupt (NMI) exception in the ADSP and flush the ADSP cache.

If the ADSP cache is flushed, the variable QURTK\_flush\_cache\_status is equal to 1; otherwise, this variable is equal to 0.



### 3.4.4 QuRT system state variable

QuRT\_System\_State is the global variable which stores the power collapse, bootup, and flush cache state status. It is updated and flushed to reflect the system state. It helps QuRT to know the state of QuRT when the system is crashed.

To view QuRT\_system\_state, in T32 follow View→Symbols→Browse. In the Browse search box, search for QuRT\_system\_state, and double-click to display the value.

The QuRT\_system\_state ranges from 0 to 21. The table below shows different states of QuRT\_system\_state. All these definitions are present in qurt\_consts.h.

On MSM8960/ APQ8064 builds, qurt\_consts.h is located at <root>\lpass\_proc\qurt\install\MMv4\include.

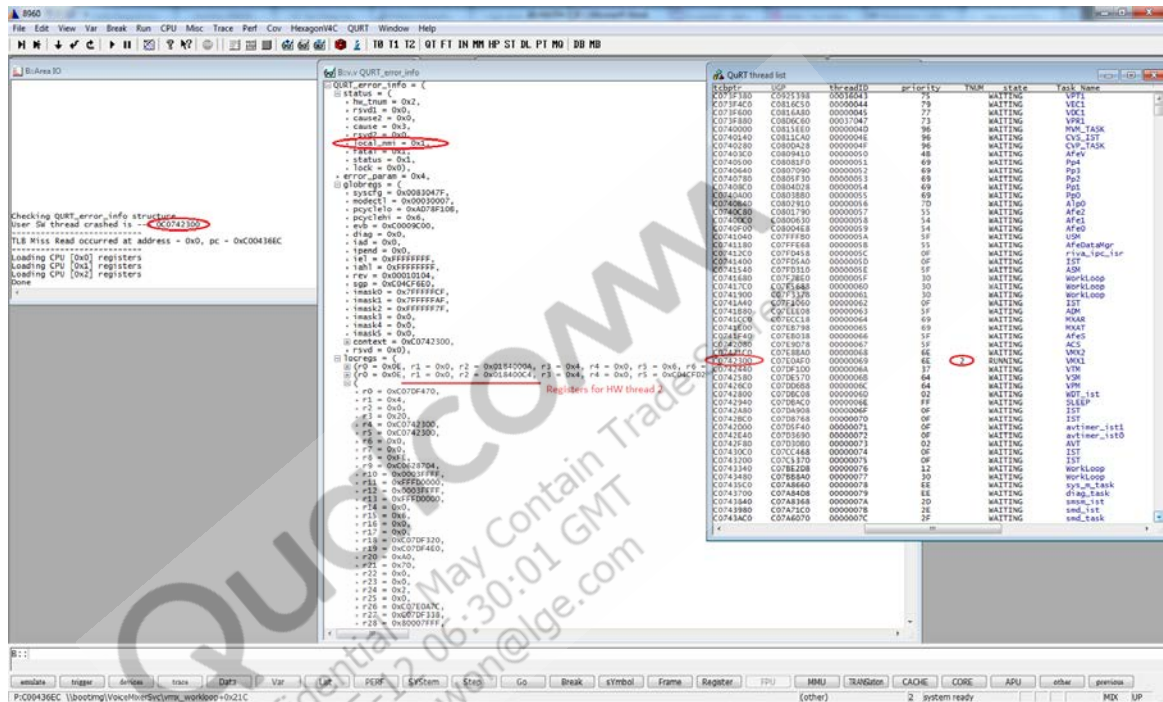
```
#define PC_PREPARE_SINGLE_MODE 1 (It means that Hexagon is entered single
threaded mode for power collapse)
#define PC_PREPARE_END 2 //Hexagon power collapse preparation is done
#define PC_PERFORM_IPEND 3 //Hexagon check IPEND register for interrupt
#define PC_PERFORM_SAVE_ISDP 4 // Hexagon power collapse - saving setting
ISDB debug registers
#define PC_PERFORM_SAVE_PMU 5 // Hexagon power collapse - save PMU before
entering power collapse
#define PC_PERFORM_SAVE_TLB 6 // Hexagon power collapse - save TLB
entries table
#define PC_PERFORM_SWITCH_PC 7 // Hexagon is in performing Power collapse
stage
#define PC_PERFORM_EXIT 8 // Hexagon is exiting power collapse)
#define PC_FLUSH_L1CACHE 9 // Hexagon flush L1 cache done
#define PC_FLUSH_L2CACHE 10 // Hexagon flush L2 cache done
#define PC_FLUSH_CACHE_DONE 11 // Hexagon flush cache done
#define PC_SWITCH_PC_DONE 12 // Hexagon is at end of performing Power
collapse stage
#define PC_BOOT_SETUP_ISDP 13 // Hexagon bootup setup ISDP register
#define PC_WB00T_INIT_TLB 14 // Hexagon warm boot - init TLB table
#define PC_WB00T_SET_1T01_MAP 15 // Hexagon warm boot - setting 1 to 1
mapping
#define PC_WB00T_CFG_ADV_SYSCFG 16 // Hexagon warm boot - setting syscfg
register
#define PC_WB00T_REMOVE_1T01_MAP 17 // Hexagon warm boot - remove 1 to 1
mapping
#define PC_CB00T_BSP_INIT 18 // Hexagon cold boot- call
__hexagon_bsp_init()
#define PC_CB00T_END_CLEAN_L1CACHE 19 //Hexagon cold or warm boot - end
clean l1 cache
#define PC_CB00T_END_CLEAN_INIT 20 //Hexagon cold or warm boot - end clean
init
#define PC_CB00T_END_OS_INIT 21 //Hexagon cold or warm boot - end boot init
```

### 3.4.5 Types of exceptions

Look at QuRT error information by typing the following command;

```
v.v QURT_error_info
```

This displays the reason for LPASS Hexagon crash. It provides which hardware thread is crashed, the reason for the Hexagon crash, and other crash related information. Try to find out from QuRT software threads which software thread maps to the crashed hardware thread.



In the example above, software thread 0xC0742300 crashed, corresponding to hardware thread 2.

QuRT error codes to distinguish from multiple errors follow.

#### Error cause codes

```
#define ERR_PRECISE 1
#define ERR_NMI 2
#define ERR_TLBMIS 3
#define ERR_RSVD_VECTOR 4
#define ERR_ASSERT 5
#define ERR_BADTRAP 6
#define ERR_TRAP1 7
#define ERR_EXIT 8
#define ERR_TLBMIS_X 10
#define ERR_STOPPED 11
```

```

1      #define ERR_FATAL_EXIT      12
2      #define ERR_INVALID_INT     13
3

```

## Error codes for SW ASSERT (cause2)

```

4
5
6      #define  ABORT_FUTEX_WAKE_MULTIPLE      1 // futex_asm.s: Abort cause -
7      futex Wake multiple
8      #define  ABORT_WAIT_WAKEUP_SINGLE_MODE 2 // power.c: Abort cause -Thread
9      waiting to wake up in Single threaded mode
10     #define  ABORT_TCXO_SHUTDOWN_NOEXIT     3 //power.c: Abort cause-tcxo
11     shutdown is call without exit
12     #define  ABORT_FUTEX_ALLOC_QUEUE_FAIL  4 // futex.c: Abort cause - Futex
13     alloc queue fail - QURTK_futexhash_lifo empty
14     #define  ABORT_INVALID_CALL_QURTK_WARM_INIT 5 // init_asm.S: Abort
15     cause - invalid Call QURTK_warm_init() in NONE CONFIG_POWER_MGMT mode
16     #define  ABORT_THREAD_SCHEDULE_SANITY    6 // switch.S: Abort cause
17     - Sanity schedule thread is not supposed to run on current hw thread
18     #define  ABORT_REMAP                     7 //Remap in the page
19     table. The right behavior should be always remove mapping if necessary
20     #define  ABORT_NOMAP                     8 //No mapping in page
21     table when remove a user mapping
22     #define  ABORT_OUT_OF_SPACES              9
23     #define  ABORT_INVALID_MEM_MAPPING_TYPE  10 //Invalid memory mapping
24     type when creating qmemory
25     #define  ABORT_NOPOOL                    11 //No pool available to
26     attach
27     #define  ABORT_LIFO_REMOVE_NON_EXIST_ITEM 12 //Can not allocate more
28     futex waiting queue
29     #define  ABORT_ARG_ERROR                  13
30     #define  ABORT_ASSERT                     14 // Assert abort
31     #define  ABORT_FATAL                     15 //FATAL error that shall
32     never happens
33

```

### 3.4.6 Determining if external NMI or self-NMI was received from dump

#### Example

```

struct QURT_error_info {
    unsigned int hw_tnum:3;    // Hardware thread number which error occurs
    unsigned int rsvd1:5;
    unsigned int cause2:8;    // use to store the abort(assert)
    reason/detail cause
    unsigned int cause:8;    // error type refer to qurt_error.h
    unsigned int rsvd2:4;
    unsigned int local_nmi:1; // it is set to 1 if NMI is triggered
    internally by Qurt
                                // it is set to 0 if NMI
    is triggered externally
    unsigned int fatal:1;    // 1 - if error is fatal, 0 - if error
    is non-fatal
    unsigned int status:1;    // 1 - used, 0 - free. Used by Qurt to
    check for first error
    unsigned int lock:1;    // spinlock status - mutex protection for
    error status
};

```

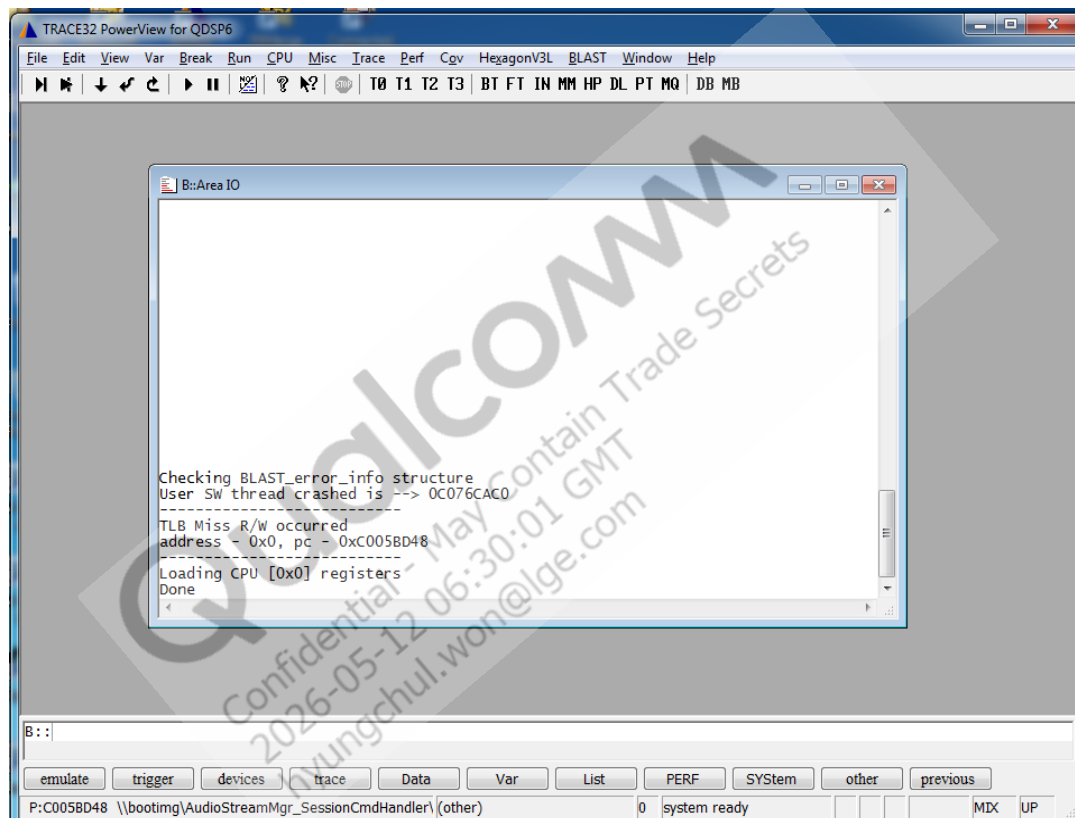
QURT\_error\_info.status.local\_nmi = 1

- 1 – NMI is triggered internally by QuRT
- 0 – NMI is triggered externally

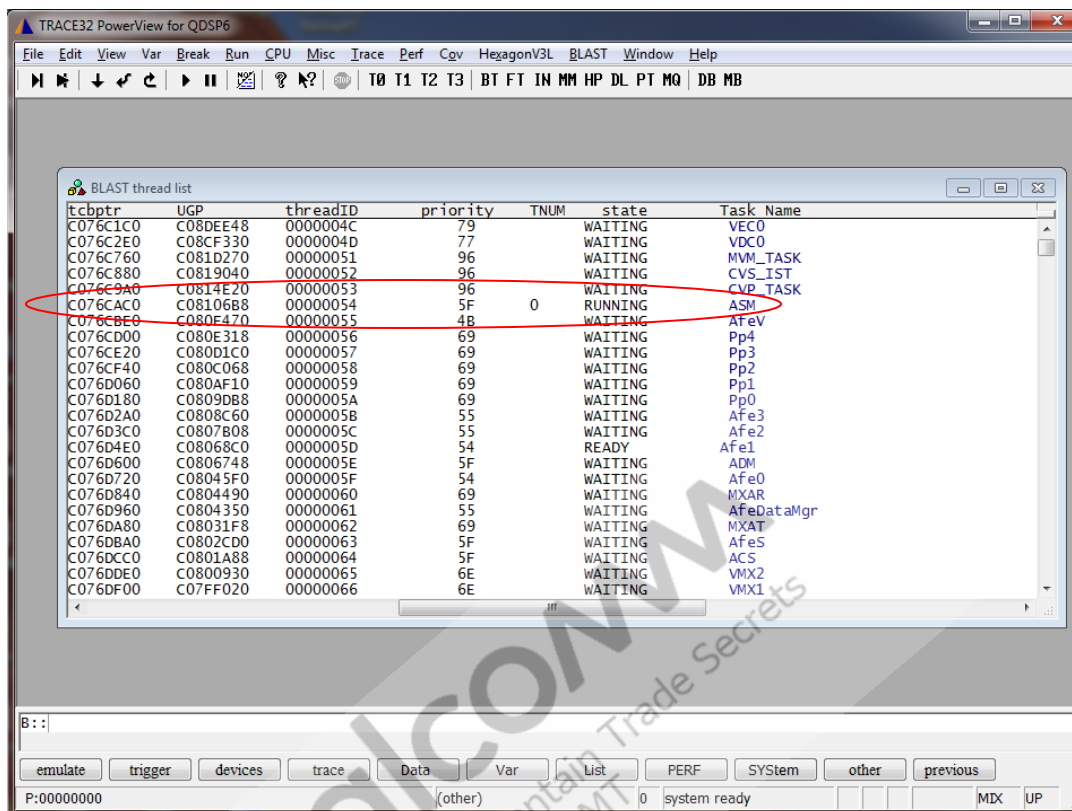
### 3.4.7 Example for debugging an exception

After an ADSP memory dump is loaded into T32 using the Crashman tool, as described in Section 3.4.2, the QURT\_error\_info structure can be viewed to determine if an exception occurred. As also described in Section 3.4.2, when an exception occurs, information regarding the exception is printed in this structure.

The screenshot shows that a TLB Miss R/W exception occurred in the software thread with tcbptr value 0xC076CAC0. This indicates that a memory access was attempted for a memory location that is unmapped in the ADSP. The pc value shown indicates that the exception occurred at the instruction with virtual address 0xC005BD48, and the address value indicates that this occurred when an attempt was made to access memory at physical address 0x0.



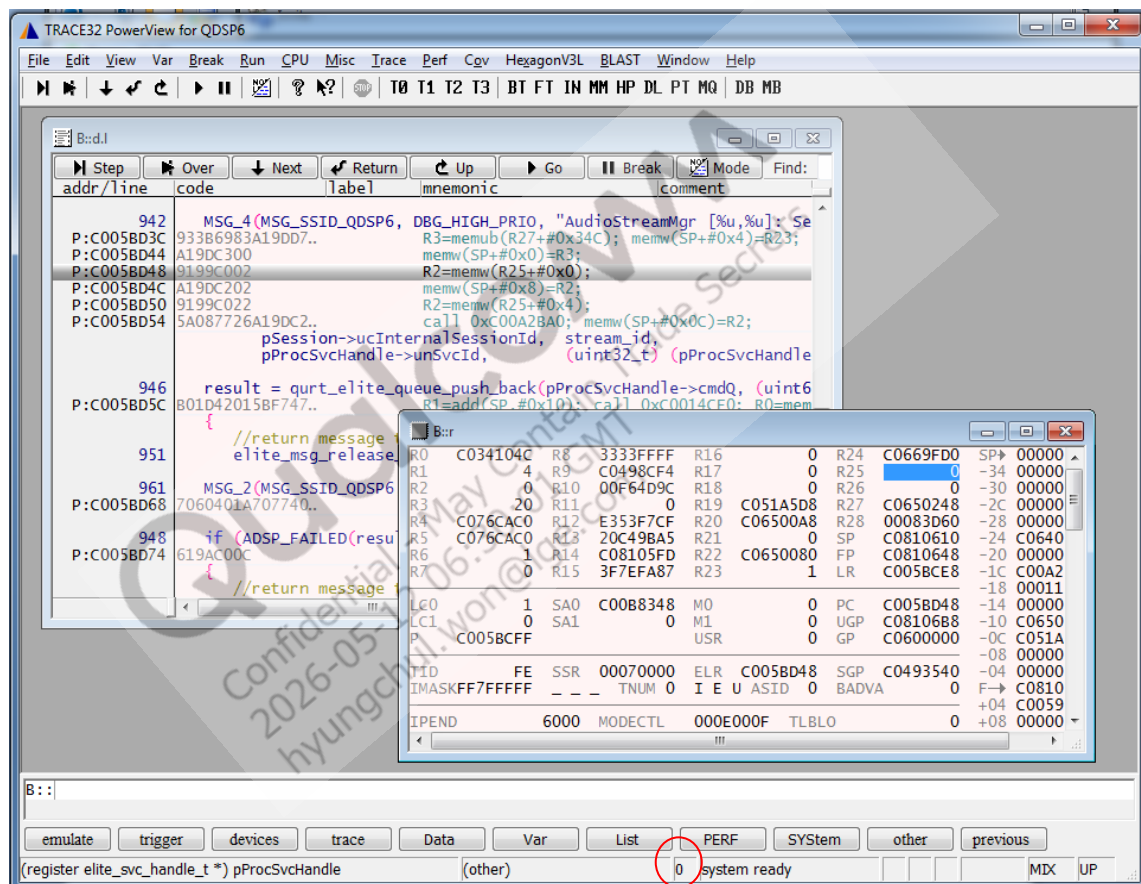
- 1 Clicking **BT** (BLAST thread list) reveals that the software thread with the tcbptr value
- 2 0xC076CAC0 is thread ASM (Audio Stream Manager) running on hardware thread 0.



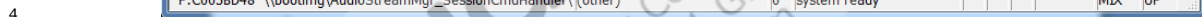
When an exception occurs, QuRT stores the register values for all running threads, along with the hardware thread number in which the exception occurred. Then, when a memory dump is loaded in T32 using the Crashman tool, the register values for the running threads are loaded and the hardware thread is switched to the same one in which the exception occurred, as shown below.

Thus, if register is entered at the T32 command prompt, the register values for the hardware thread in which the exception occurred are displayed. Register ELR holds the address of the instruction in which the exception occurred. If data.list is entered, this instruction is displayed, highlighted in gray in its source file.

This screenshot shows that ELR is equal to 0xC005BD48 (as shown in the QURT\_error\_info structure), and the instruction at this address is also displayed. This instruction is `R2=memw(R25+#0x0);`, which is a 32-bit read from memory from the address pointed to by R25, into R2. The register list indicates that R25 is equal to 0, so that this is read from address 0x0.



The following shows the pagetable information, where address 0x0 is not mapped in the pagetable. When an attempt is made to access this memory, a TLB Miss R/W exception occurs. The next step for debugging would be to determine how register R25 was set to 0.



### 5 3.4.8 On MSM8974

On MSM8974, as the ADSP contains both Audio-Voice framework and sensors, the first level of triage can help to get the right team. After loading the dumps and seeing QuRT Thread list, if the software or hardware thread that crashed is a sensors thread, this would indicate the origin of the crash. In general, sensors threads are preceded by SNS\_, e.g., SNS\_SMGR, SNS\_DEBUG etc. See [Q6] and [Q7] for sensors debugging techniques.

|    |          |          |          |    |   |         |      |
|----|----------|----------|----------|----|---|---------|------|
| 11 | F0018080 | FOA34F60 | 00000064 | 5F | 0 | WAITING | ACS  |
|    | F0018160 | FOA33DF0 | 00000065 | 5F | 0 | WAITING | VMX2 |



### 3.5 Debugging bootup issues

Flash NAND is used to store the ADSP image .elf file contents. At device bootup, the sections from this .elf file are then copied to EBI (RAM) memory at the physical addresses specified in the .elf file. The physical addresses and locations of sections within the .elf file can be seen by entering the command:

```
qdsp6-objdump -x <ADSP image>
```

Example output for this command is shown below. The physical start address of ADSP memory in EBI for this example is 0x8DA00000. The physical start addresses, sizes of sections, and locations of sections within an .elf file are specified under the Sections heading, under columns LMA, Size, and File off respectively. For example, the .rodata section starts at physical address 0x8DD3F000, has size 0x18D07E, and is at location (byte offset) 0x340000 within the .elf file.

```
dsp.elf:      file format elf32-littleqdsp6
dsp.elf
architecture: qdsp6v4, flags 0x00000102:
EMSG-P, D-PAGEB
start address 0x8da00000
Program Header:
LOAD off 0x00001000 vaddr 0x28400000 paddr 0x28400000 align 2**12
  file off 0x00000000 memsz 0x00020000 flags rw-
LOAD off 0x00001000 vaddr 0xc0000000 paddr 0x8da00000 align 2**12
  file off 0x00033edd0 memsz 0x00033edd0 flags r-x
LOAD off 0x00034000 vaddr 0xc0033f000 paddr 0x8dd3f000 align 2**12
  file off 0x0018d07e memsz 0x0018d07e flags r-
LOAD off 0x004ce000 vaddr 0xc04ce000 paddr 0x8dece000 align 2**12
  file off 0x000b87ac memsz 0x000b87ac flags rw-
LOAD off 0x00587000 vaddr 0xc0587000 paddr 0x8e000000 align 2**12
  file off 0x00003440 memsz 0x0018b180 flags rw-
Sections:
Idx Name      Size      LMA      File off  Algn
0 .start      00000008 c0000000 cda00000 00000000 2**5
CONTENTS, ALLOC, LOAD, READONLY, CODE
1 .init       00000054 c0001000 8da01000 00002000 2**5
CONTENTS, ALLOC, LOAD, READONLY, CODE
2 .text       0033dc90 c0001100 8da01100 00002100 2**8
CONTENTS, ALLOC, LOAD, READONLY, CODE
3 .fini       00000030 c033eda0 8dd3eda0 0033fda0 2**5
CONTENTS, ALLOC, LOAD, READONLY, CODE
4 .rodata     0018d07e c033f000 8dd3f000 00340000 2**10
CONTENTS, ALLOC, LOAD, READONLY, DATA
5 .data       000b6ef0 c04ce000 8dece000 004ce000 2**12
CONTENTS, ALLOC, LOAD, DATA
6 .eh_frame   00001734 c0584f00 8df84f00 00584f00 2**5
CONTENTS, ALLOC, LOAD, READONLY, DATA
7 .gcc_except_table 00000151 c0586634 8df86634 00586634 2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
8 .ctors      0000001c c0586788 8df86788 00586788 2**2
CONTENTS, ALLOC, LOAD, DATA
9 .dtors      00000008 c05867a4 8df867a4 005867a4 2**2
CONTENTS, ALLOC, LOAD, READONLY, DATA
10 .sdata     00003440 c0600000 8e000000 00587000 2**5
CONTENTS, ALLOC, LOAD, DATA
11 .sbss      00000040 c0603440 8e003440 0058a440 2**3
ALLOC
12 .lpm_decoder 00010000 28400000 28400000 00001000 2**3
ALLOC
13 .lpm_audifdma 00003000 28410000 28410000 00001000 2**3
ALLOC
14 .lpm_audio  0000d000 28413000 28413000 00001000 2**3
ALLOC
15 .bss       00187180 c0604000 8e004000 0058a440 2**12
ALLOC
16 .comment   000000bf 00000000 00000000 0058a440 2**0
CONTENTS, READONLY
SYMBOL TABLE:
no symbols
```

When the ADSP is taken out of reset during device bootup, it begins executing instructions starting at the physical address specified by register QDSP6SS\_RST\_EVB described in Section 2.5. Before taking the ADSP out of reset, the ADSP voltage (described in Section 2.4) must be set to 1.1 V and the PXO clock must be on, since the ADSP first runs on PXO. For MSM8660, PLL4 is also voted for through RPM before the ADSP is taken out of reset.

When taken out of reset, the ADSP first performs QuRT initialization, and during this initialization the ADSP clock is switched from PXO to PLL4 (which runs at a higher frequency than PXO). For MSM8660, after switching to PLL4, the ADSP then votes for PLL4 through RPM. For MSM8960/APQ8064, PLL4 is enabled through a hardware register, and no vote through RPM is required for PLL4. Thus, for MSM8960/APQ8064 during QuRT initialization, to enable PLL4 the ADSP writes value 1 to bit 4 of hardware register PLL\_ENA\_LPASS\_DSP at physical address 0x00903520, then after this the ADSP switches from PXO to PLL4.

For MSM8660, the vote for PXO and PLL4 from the Linux kernel to RPM is removed after 10 sec, so that the ADSP must vote for and switch to PLL4 within this time. For the MSM8960/APQ8064, the vote for PXO from the Linux kernel to RPM is removed after 10 sec, so that the ADSP must enable and switch to PLL4 within this time.

After QuRT initialization is complete, function main() is entered. At main() there are further initializations, including heap, Shared Memory Driver (SMD), Elite framework, and Multimedia Power Management (MMPM) initialization. More information regarding the bootup sequence on MSM8960/APQ8064 is provided in Section 6.1.

After SMD initialization completes during ADSP bootup, ADSP then sets the SMSM bit, which is acknowledged by the Linux kernel and used as confirmation that the ADSP has successfully booted up. The Linux kernel then prints the following message in the Linux kernel logs:

```
... apr_tal:Q6 Is Up
```

Thus, if this message is not shown in the Linux kernel logs, this indicates that the ADSP has not booted up as expected. Typical reasons for the ADSP not booting up as expected are:

- The ADSP image sections have been loaded to incorrect physical memory addresses.
- ADSP voltage and/or clocks are not set as expected before the ADSP is taken out of reset.
- An issue has occurred in the ADSP, e. g., an exception has occurred, which prevents it from completing its bootup procedure.

An ADSP memory dump is needed to debug ADSP bootup issues. See Section 3.1 for instructions on obtaining an ADSP memory dump, and Section 3.4.2 for instructions on loading the dump in T32. Note that if QuRT initialization has not completed, there may be no threads displayed in the QuRT thread list (described in Section 2.2.4), and the extracted logs, e.g., F3, APR logs, etc., may be empty if no log messages were printed yet. Also, if the QuRT error handler has not been initialized yet, errors and exceptions may not appear in the QuRT\_error\_info structure (described in Section 3.4.2).

Methods to debug ADSP bootup issues are:

- When the ADSP reaches main(), it copies the Q6\_BUILD timestamp to a second location. Use the procedure in Section 3.4.1 to determine if this timestamp appears multiple times in the memory dump. If it does, this indicates that the ADSP successfully reached main() but stopped running before initialization completed.
- Check the value of variable qurt\_has\_initted. When QuRT initialization completes, this variable is set to 1. Otherwise, its value is 0.
- Check the value of variable QURT\_system\_state. Within the first 60 instructions of ADSP execution, the value of this variable is changed from 0 to a nonzero value; if this variable is equal to 0, it may indicate that the ADSP never started running.

- Ensure the ADSP image sections have been loaded at the correct physical addresses, using the information in the ADSP image .elf file, shown as an example in the above screenshot. Also check for memory corruption by following the instructions in Section 3.7.
- Measure ADSP voltage (as described in Section 2.4) before the ADSP is taken out of reset and confirm it is at 1.1 V as expected. After the ADSP has completed initialization, and if it has received no commands, it will enter power-collapse and the ADSP voltage will be lowered. For MSM8660, the ADSP voltage during power-collapse is 0.35 V, and for MSM8960/APQ8064 the ADSP voltage during power collapse is 0 V.
- Determine if the ADSP is running on PLL4 as expected.
  - For MSM8660, check the value of hardware register LCC\_Q6\_FUNC at physical address 0x2800001c, as described in Section 2.5.2. If bits 4:3 of this register are equal to 0b00, this indicates that the ADSP is still running on PXO and has not reached the point in QuRT initialization to switch to PLL4. If these bits are equal to 0b01, this indicates that the ADSP has switched to PLL4 as expected.
  - For MSM8960/APQ8064, check the value of register PLL\_ENA\_LPASS\_DSP at physical address 0x00903520 and ensure bit 4 is equal to 1. This indicates that the ADSP has enabled PLL4. Next, check the value of hardware register LCC\_PLL0\_STATUS at physical address 0x28000018, as described in Section 2.5.3. If PLL4 has been enabled and is locked, this register is equal to 0x10001.

### 3.5.1 Checking boot-up process from RAM-dumps

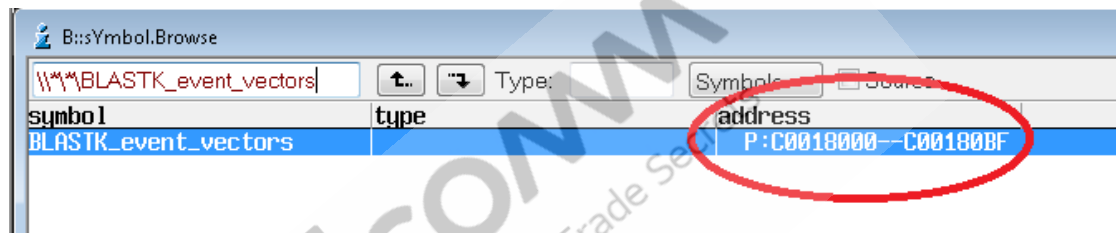
Another way to determine to which point the boot-up has come, load RAM-dumps into T32 and check for value <QURT\_system\_state>. This value is updated from function QURTOS\_init in crt0.s and file qurt\_consts.h defines all the values that it can take. Value between 14 and 21 are used during bootup and depending on the value one can know the stages the bootup process has reached.

## 3.6 Checking for memory corruption

1. Load the memory dump in T32.
2. Determine the address range of the sections you want to compare.
3. Enter the following:

```
d.load.elf dsp.elf a:<start physical address>--<end physical address>
/compare
```

Note that when comparing the .text section, variable QURTK\_event\_vectors (the jump table for interrupts) will usually be different due to the RTOS configuration, so this table should not be compared. As can be seen below, variable QURTK\_event\_vectors is in virtual address range 0xC0018000 to 0xC00180BF, which corresponds to physical address range 0x46718000 to 0x467180BF (see Section 6.2 for information on how to determine virtual-to-physical address mapping).



4. Run the DSP connect script (DSP\_connect.cmm) again to load the symbols.

### Example of comparing part of text region, i.e., the .text section

1. Run symbol.list.section to find the sections that are Read-Execute (R-X)/Read (R)-only. This is displayed below. For this example, virtual address 0xC0000000 corresponds to physical address 0x46700000.

The screenshot shows the 'BssSymbol.List.SECTION' window. It displays a table of sections with columns: address, to, path, section, acc, init, and physical. The section 'bootimg\text' is highlighted with a red box.

| address              | to | path     | section              | acc | init | physical |
|----------------------|----|----------|----------------------|-----|------|----------|
| D:28400000--2840FFFF |    | bootimg\ | lpm_decoder          | RW- | L-   |          |
| D:28410000--28412FFF |    | bootimg\ | lpm_audifdma         | RW- | L-   |          |
| D:28413000--284197FF |    | bootimg\ | lpm_audio            | RW- | L-   |          |
| D:28419800--2841DE30 |    | bootimg\ | lpm_audio_static_svc | RW- | L-   |          |
| D:C0000000--C0000C3F |    | bootimg\ | start                | R-X | L-   |          |
| P:C0000C40--C0000C9F |    | bootimg\ | init                 | R-X | L-   |          |
| P:C0001000--C0314C4F |    | bootimg\ | text                 | R-X | L-   |          |
| P:C0314C60--C0314C9F |    | bootimg\ | fini                 | R-X | L-   |          |
| D:C0315000--C04646EF |    | bootimg\ | rodata               | R-- | L-   |          |
| D:C0466000--C0513C1F |    | bootimg\ | data                 | RW- | L-   |          |
| D:C0513C20--C0514C93 |    | bootimg\ | eh_frame             | RW- | L-   |          |

2. Compare the .text section in the virtual address range 0xC0001000 to 0x C0017FFF (before the jump table for interrupts), which corresponds to physical address range 0x46701000 to 0x46717FFF.

```
d.load.elf dsp.elf a:0x46701000--0x46717fff /compare
```

3. Compare the .text section in the virtual address range 0xC00180C0 to 0xC0314C4F (after the jump table for interrupts), which corresponds to physical address range 0x467180C0 to 0x46A14C4F.

```
d.load.elf dsp.elf a:0x467180c0--0x46a14c4f /compare
```

If output gives no error, the text sections are the same. Otherwise if a difference is detected, the first mismatching address will be printed out.

In order to provide an example of detected memory corruption, QTI overwrites the instruction at virtual address 0xC0314C10 (this instruction is `jmprr LR`, encoded as 0x529FC000).

|            |          |               |
|------------|----------|---------------|
| D:C0314C0C | 901EC01E | deallocframe; |
| D:C0314C10 | 529FC000 | jmprr LR;     |

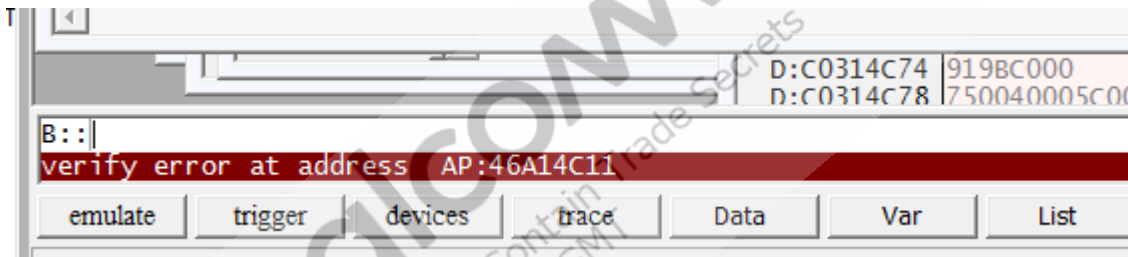
This instruction is overwritten so that it is equal to 0.

D.S D:0xC0314C10 %LE %LONG 0x0

The resulting memory is shown here.

|            |          |               |
|------------|----------|---------------|
| D:C0314C0C | 901EC01E | deallocframe; |
| D:C0314C10 | 00000000 |               |

When the above compare commands are entered, an error is printed in T32.

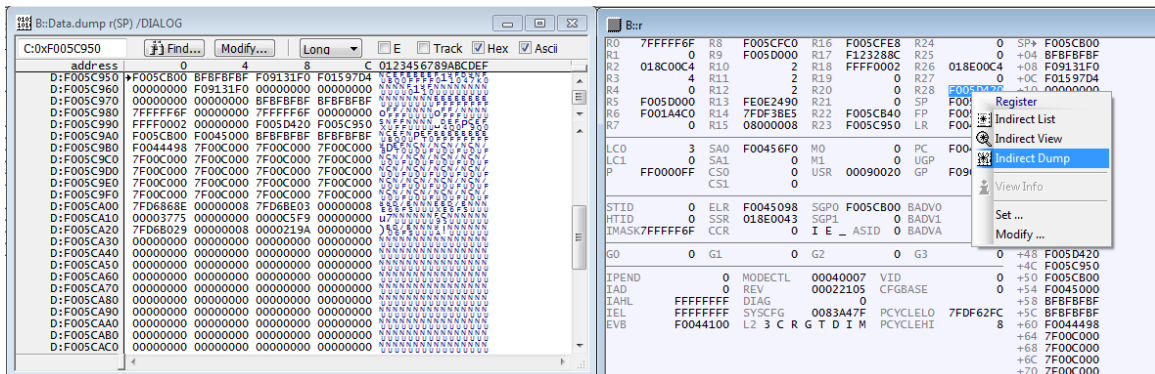


This shows that memory corruption was detected at physical address 0x46A14C11, which corresponds to virtual address 0xC0313C11 (the memory corruption starts at the second byte of the instruction).

### 3.7 Stack frame analysis

Stack frames would indicate the memory stack in use, its origin and many other details of the function being executed.

In T32, right-clicking a pointer, such as **SP**, and selecting **Indirect Dump** would allow us to view the content of that memory location.





For example, if function1() is being executed and calls function2(), then at function2 call, a new stack location is reserved for function2(). Each stack frame contains a Stack Pointer (SP) and Frame Pointer (FP) along with the needed stack size. Each function's stack is between SP and FP, e.g., the circled data is the stack frame of a particular function.

B:r

|       |           |          |           |          |          |       |          |     |          |          |
|-------|-----------|----------|-----------|----------|----------|-------|----------|-----|----------|----------|
| R0    | 6CB16749  | R8       | 0         | R16      | F09B51B0 | R24   | 4        | SP  | 00000024 |          |
| R1    | 0B        | R9       | 0         | R17      | 24       | R25   | 2DA905D0 | -0C | F06A056C |          |
| R2    | FE2A0000  | R10      | 2DA29AEC  | R18      | 1        | R26   | 2DAE89AD | -08 | F09B51B0 |          |
| R3    | 0382E394  | R11      | F06AAB74  | R19      | F06A056C | R27   | 0        | -04 | 00000005 |          |
| R4    | 0381E3A0  | R12      | 3         | R20      | 2DA9AB38 | R28   | 10       | F   | F0078A70 |          |
| R5    | 2DA5326E  | R13      | 1         | R21      | 1        | SP    | F0078A00 | +04 | F0186764 |          |
| R6    | 0F0D8130  | R14      | F017DC70  | R22      | 2DA905BC | FP    | F0078A10 | +08 | F0078A28 |          |
| R7    | 2D        | R15      | FF        | R23      | F005C090 | LR    | F01864C4 | +0C | F0250EC4 |          |
|       |           |          |           |          |          |       |          |     | +10      | 00000000 |
| LC0   | 1         | SA0      | 2DA2C480  | M0       | 0        | PC    | F015A0D8 | +14 | 0F0DB868 |          |
| LC1   | F0016280  | SA1      | 0         | M1       | 0        | UGP   | F0078B40 | +18 | F0078A78 |          |
| P     | FFFFFFF00 | CS0      | 0         | USR      | 00090020 | GP    | 2DAFF000 | +1C | F0251460 |          |
|       |           |          |           |          |          |       |          |     | +20      | F0078A40 |
|       |           |          |           |          |          |       |          |     | +24      | F1265148 |
| STID  | 0         | ELR      | F015A0D8  | SGP0     | F005C090 | BADV0 | 0        | +28 | F007B560 |          |
| HTID  | 0         | SSR      | 018F0143  | SGP1     | 0        | BADV1 | 0        | +2C | 00000001 |          |
| IMASK | FFFFFFFDF | CCR      | 0         | I E U    | ASID 1   | BADVA | 0        | +30 | 00000001 |          |
|       |           |          |           |          |          |       |          |     | +34      | 00000005 |
| G0    | 0         | G1       | 0         | G2       | 0        | G3    | 0        | +38 | 00000007 |          |
|       |           |          |           |          |          |       |          |     | +3C      | 00000000 |
| IPEND | 0         | MODECTL  | 00040007  | VID      | 0        |       | 0        | +40 | 0000002D |          |
| IAD   | 0         | REV      | 00022105  | CFGBASE  | 0        |       | 0        | +44 | 00000001 |          |
| IAHL  | FFFFFFFF  | DIAG     | 0         |          |          |       |          | +48 | 2DA9AB38 |          |
| IEL   | FFFFFFFF  | SYSCFG   | 0083A47F  | PCYCLELO | 7FDF62FC |       |          | +4C | 00000001 |          |
| EVB   | F0044100  | L2 3 C R | G T D I M | PCYCLEHI | 8        |       |          | +50 | 2DA9ADC0 |          |
|       |           |          |           |          |          |       |          |     | +54      | 00000000 |

The address of SP is 0xF0078A00 containing value 0x24. The address of FP is 0xF0078A10 containing value 0xF0078A70. FP contains the SP of a previous frame, in our example SP of function1(). Helping the program to trace back to the previous function once function2() completes its execution, at which point function2's stack is no longer in use. For more information on general purpose registers on Hexagon, refer to the Hexagon user guide that is part of the Hexagon\_Tools installation.

### 3.7.1 Scripts to reconstruct callstacks

This is a default step while using the Crashman tool.

## 4 Simulator Debugging

---

This chapter discusses T32 debugging with a test framework.

### On MSM8660

1. Open the command prompt and CD to the test framework root directory.
2. Type set HEXAGON\_IMAGE\_ROOT=<ADSP Framework build directory>.
3. Type adsptest-t32.cmd.

### On MSM8960/APQ8064

1. Open the command prompt to the <ADSP Framework root>/lpass\_proc/adsptest\_cosim directory of the ADSP Framework.
2. Type adsptest-t32.cmd.

Qualcomm  
Confidential - May Contain Trade Secrets  
2026-05-12 06:30:01 GMT  
hyungchul.won@lge.com

# 5 General Debugging

## 5.1 Hexagon log analysis

This chapter analyzes the log messages and what those log messages mean in detail. The F3 logs are the last 128 kB of messages stored in the Hexagon log memory when the crash has occurred. Before analyzing the messages, it is advised to become familiar with the Elite framework and associated interface specification documents. See [Q3] for the required documents.

The log messages look something like this:

```
00:10:58.050      ./src/AudioStreamMgr_AprIf.cpp 00085 AudioStreamMgr:
Rec cmd 0x10bcb at [Addr=0x407, Port=0x101]
```

The message's first field is the time from the DSP clock (00:10:58.050) at which the message was created. The second and third fields are the file and line from the file from which the message was printed (./src/AudioStreamMgr\_AprIf.cpp 00085). The fourth and rest of the fields are the actual message.

The above message tells that the Audio Stream Manager (ASM) has received a command with the ID 0x10bcb with address and ports as displayed. As the message was displayed from the ASM, refer to the Audio ISOD and/or ASM API reference guides to know what the command 0x10bcb means. Per [Q3], 0x10bcb refers to ASM\_STREAM\_CMD\_OPEN\_READ.

## 5.2 Hexagon messages for audio record

Following the above description, from the phone UI, when the user selects an appropriate application for audio recording, the apps processor sends required commands to set up the topology and modules for the specified task. All communication between the apps processor and Hexagon is over shared memory and using the Asynchronous Packet Router (APR). See Section 3.2 of [Q4] for the complete sequence flow.

1. The apps processor tells Hexagon that a record session will take place, and also informs Hexagon with the record format. This part of the message is handled by the static service ASM.

```
00:10:58.080      ./src/AudioStreamMgr_AprIf.cpp 00085 AudioStreamMgr:
Rec cmd 0x10bcb at [Addr=0x407, Port=0x101]
00:10:58.080      ../../../../datalink/aprtal/src/aprtalsmd.c 00300
apr_tal_smd_rx_cb: bytes_remaining 36, size 0
00:10:58.080      ./src/AudioStreamMgr.cpp 00342 Signal received = 4 !
00:10:58.080      ./src/AudioStreamMgr_SessionCmdHandler.cpp 00101
AudioStreamMgr [0,1]: receive apr command 0x10bcb at port [0x 101]
00:10:58.080      ./src/AudioStreamMgr_SessionCmdHandler.cpp 00455
```



```

1   AudioStreamMgr [0,1]: Record Format 0x10be5
2   00:10:58.080      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00468
3   AudioStreamMgr [0,1]: Creating Pre-Proc Service

```

The command 0x10bxb is the ASM\_STREAM\_CMD\_OPEN\_READ and 0x10be5 is the record format, which is Linear PCM.

2. After receiving the message, the ASM creates and connects required encoder and preprocessing modules per this request:

```

10  00:10:58.081      ./src/audproc_appi_init.cpp  01052  PAB38 audproc_svc:
11  Initializing poprep Begin
12  00:10:58.081      ./src/appi_genericresampler.cpp  00117  Generic
13  Resampler APPI Init
14  00:10:58.081      ./src/appi_softvolumecontrols.cpp  00099  Soft Volume
15  Controls APPI Init
16  00:10:58.081      ./src/appi_softvolumecontrols_utils.cpp  00041
17  audproc_svc: SVC left channel gain setting in linear scale 8192
18  00:10:58.081      ./src/appi_softvolumecontrols_utils.cpp  00042
19  audproc_svc: SVC right channel gain setting in linear scale 8192
20  00:10:58.081      ./src/appi_softvolumecontrols_utils.cpp  00060
21  audproc_svc: SVC master gain setting in linear scale 8192
22  00:10:58.082      ./src/audproc_appi_init.cpp  01150  PAB38 audproc_svc:
23  Initializing poprep End
24  00:10:58.082      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00506
25  AudioStreamMgr [0,1]: PP Service [0xe0000,0xa04d] Created

```

3. The created preprocessing modules are assigned ID 0xe0000.

4. Per the requested encoder, the required encoder service is created next, with the ID 0x170000.

```

31  00:10:58.082      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00506
32  AudioStreamMgr [0,1]: PP Service [0xe0000,0xa04d] Created
33  00:10:58.082      ./src/AudioStreamMgr_Util.cpp  00615  AudioStreamMgr
34  [0,1]: Creating Encoder Service
35  00:10:58.082      ./src/Hexos_Queue.cpp  00319  Q CREATE: Q=0xc066e8a8
36  NumItems=16
37  00:10:58.082      ./src/Hexos_Queue.cpp  00319  Q CREATE: Q=0xc066e968
38  NumItems=8
39  00:10:58.082      ./src/Hexos_Queue.cpp  00319  Q CREATE: Q=0xc066e9e8
40  NumItems=8
41  00:10:58.083      ./src/AudioEncSvc_Util.cpp  00128  Creating an encoder
42  with ID 0x10be5
43  00:10:58.083      ./src/CEncoderCapiLib.cpp  00149
44  VoiceEncoderCapiType::VCAPI_Init()
45  00:10:58.083      ./src/CEncoderCapiLib.cpp  00264
46  VoiceEncoderCapiType::VCAPI_ReInit()
47  00:10:58.083      ./src/Hexos_Thread.cpp  00150  THRD CREATE:
48  Thread=0xa04b Name(Hex)= 41, 45, 6e, 63, 0, 0, 0, 0

```

```

1      00:10:58.083      ./src/AudioEncSvc.cpp  00196  AudioEncSvc creation
2      success!!
3      00:10:58.083      ./src/AudioStreamMgr_Util.cpp  00682  AudioStreamMgr
4      [0,1]: Encoder Service [0x170000,0xa04b] Created
5      00:10:58.083      ./src/AudioStreamMgr_DevIF.cpp  00090  AudioStreamMgr
6      [0,1]: Request TX Handle

```

5. The Audio Device Manager (ADM) creates device-specific modules to connect with the encoder and preproc modules that are created by the ASM. Along with creating the services, the required queues are also created.

```

12     00:10:58.083      ./src/AudDevMgr.cpp  00314  ADM: rcvd custom msg
13     [opcode] = [0x300003]
14     00:10:58.083      ./src/AudDevMgr.cpp  00526  As part of Tx Stream
15     Connect, issuing CONNECT_MT_MX_OUT_PORT cmd

```

Code 0x300003 refers to ELITEMSG\_CUSTOM\_ADM\_STREAM\_CONNECT. This connects with the stream services created by ASM.

The module, which connects the modules created by ASM with modules created by ADM, is a matrix service. The commands which configure the matrix service are:

```

22     00:10:58.083      ./src/MixerSvc.cpp  00337  MtMx #1: rcvd custom msg
23     0x180002
24     00:10:58.083      ./src/MixerSvc_MsgHandlers.cpp  01345  MtMx #1:
25     Processing cmd 0x180002 [port ID, port cfg] = [-1, 1]
26     00:10:58.083      ./src/MixerSvc_MsgHandlers.cpp  01368  MtMx #1 creating
27     new output port 0
28     00:10:58.085      ./src/MixerSvc_MsgHandlers.cpp  01400  MtMx #1: [o/p
29     port id, num_chan] = [0, 2]
30     00:10:58.085      ./src/MixerSvc_MsgHandlers.cpp  01405  MtMx: channel
31     mapping = 1, 2, 0, 0, 0, 0, 0, 0
32     00:10:58.085      ./src/Hexos_Queue.cpp  00096  Q CREATE: Q=0xc067aab0
33     NumItems=4
34     00:10:58.085      ./src/MixerSvc_MsgHandlers.cpp  01694  MtMx #1: Leaving
35     cmd [0x180002] handler with status 0x0
36     00:10:58.085      ./src/AudioStreamMgr.cpp  00342  Signal received = 8 !
37     00:10:58.085      ./src/AudioStreamMgr_Session.cpp  00600  AudioStreamMgr
38     [0,1]: connecting Svc [0xe0000,0xa04d] to Svc [0x170000,0xa04b]!

```

Code 0x180002 is ELITEMSG\_CUSTOM\_CFG\_OUTPUT\_PORTS, which configures the output port. For the record session, the output port of a matrix is into ASM.

6. The Audio Front End (AFE) configures the audio I/O port; in this example it configures the mic.

```

45     00:10:58.130      ./src/AFESvcAprIf.cpp  00083  At <0x404> APRCMD <0x
46     100d3> AfeSvc: Rec cmd.

```

See [Q3]; code 0x100d3 means AFE\_PORT\_AUDIO\_IF\_CONFIG. A few device configurations are:

```
00:10:58.135      ./src/AFEPortAprHandler.cpp 00087 AFECmdDmaStart:
dir=1, sample_rate=48000, nChannels=1, Gain=0x2000
00:10:58.135      ./src/AFEVfrDriver.cpp 00115 added port
port_id=0,dir=1 to vfr sync
00:10:58.135      ./src/AFEPortAprHandler.cpp 00113 AFECmdDmaStart: Dma
Start Success: dir=1, intf=0.
```

#### 7. The ADM creates the COPP session:

```
00:10:58.173      ./src/AudDevMgr_AprIf.cpp 00084 At 0x4080001 APRCMD
0x10304 ADM rcvd payload size 20
00:10:58.173      ../../../../datalink/aprtal/src/aprtalsmd.c 00300
apr_tal_smd_rx_cb: bytes_remaining 40, size 0
00:10:58.173      ./src/AudDevMgr.cpp 00241 ADM: rcvd APR msg [opcode,
dst port] = [0x10304, 0x1]
00:10:58.173      ./src/AudDevMgr.cpp 01088 ADM processing
ADM_CMD_COPP_OPEN
00:10:58.173      ./src/AudDevMgr.cpp 01424 As part of Tx
ADM_CMD_COPP_OPEN, issuing CREATE_COPRep cmd
00:10:58.173      ./src/audproc_svc.cpp 00196 audproc_svc: Creating
00:10:58.173      ./src/audproc_paramhandler.cpp 00344 P0 audproc_svc:
Set output media Begin: keepInputFreq: 1, outputFreq: 0,
useNativeNumChannels: 1, outputChannels: 0
00:10:58.173      ./src/audproc_paramhandler.cpp 00396 audproc_svc:
Setting output media End
00:10:58.173      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc067b358
NumItems=8
00:10:58.173      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc06704d0
NumItems=64
00:10:58.173      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc0670530
NumItems=8
00:10:58.173      ./src/audproc_appi_init.cpp 00515 PCD3B audproc_svc:
Initializing coprep Begin
00:10:58.173      ./src/audproc_appi_init.cpp 00600 audproc_svc:
Requesting 105448 bytes of memory for coprep!.
```

8. The command 0x10304 is ADM\_CMD\_COPP\_OPEN, and COPP is the module in between AFE and matrix services. In addition, required queues and the memory required for those queues are created. The device-side calibrations, such as volume control and other COPP modules such as MBIIR, are created as shown in the following:

```
00:10:58.173      ./src/appi_softvolumecontrols.cpp 00099 Soft Volume
Controls APPI Init
00:10:58.175      ./src/appi_softvolumecontrols_utils.cpp 00041
audproc_svc: SVC left channel gain setting in linear scale 8192
00:10:58.175      ./src/appi_softvolumecontrols_utils.cpp 00042
```

```

1      audproc_svc: SVC right channel gain setting in linear scale 8192
2      00:10:58.175      ./src/appi_softvolumecontrols_utils.cpp 00060
3      audproc_svc: SVC master gain setting in linear scale 8192
4      00:10:58.175      ./src/appi_multibandir.cpp 00116 MultiBandIIR APPI
5      Init for 3 bands
6      00:10:58.175      ./src/appi_multibandir.cpp 00331 MultiBandIIR APPI
7      set_param
8      00:10:58.175      ./src/appi_eans.c 00118 EANS APPI Init
9      00:10:58.175      ./src/appi_multibandir.cpp 00116 MultiBandIIR APPI
10     Init for 5 bands
11     00:10:58.175      ./src/appi_mbdrc.cpp 00121 MBDRC APPI Init
12     00:10:58.175      ./src/audproc_appi_init.cpp 00750 PCD3B audproc_svc:
13     Initializing coprep End
14

```

#### 9. COPP connects to the matrix.

```

15
16
17     00:10:58.175      ./src/AudDevMgr.cpp 01454 As part of Tx
18     ADM_CMD_COPP_OPEN, issuing CONNECT_MT_MX_IN_PORT cmd
19     00:10:58.175      ./src/MixerSvc.cpp 00337 MtMx #1: rcvd custom msg
20     0x180001
21     00:10:58.175      ./src/MixerSvc_MsgHandlers.cpp 01084 MtMx #1:
22     Processing cmd 0x180001 [port ID, port cfg] = [-1, 1]
23     00:10:58.175      ./src/MixerSvc_MsgHandlers.cpp 01108 MtMx #1 creating
24     new input port 0
25     00:10:58.175      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc0672b70
26     NumItems=8
27     00:10:58.175      ./src/MixerSvc_MsgHandlers.cpp 01320 MtMx #1: Leaving
28     cmd [0x180001] handler with status 0x0
29     00:10:58.175      ./src/AudDevMgr.cpp 01462 As part of Tx
30     ADM_CMD_COPP_OPEN, issuing CONNECT_COPP cmd
31     00:10:58.175      ./src/audproc_svc.cpp 00834 PCD3B audproc_svc:
32     instance 0xC0670340 connecting to SvcID 0x f0000
33     00:10:58.175      ./src/audproc_svc.cpp 00926 PCD3B audproc_svc: COPReP
34     Connection Succeeded.
35     00:10:58.175      ./src/AudDevMgr.cpp 01467 As part of Tx
36     ADM_CMD_COPP_OPEN, issuing AFE_CONNECT_REQ cmd
37

```

Command 0x180001 is ELITEMSG\_CUSTOM\_CFG\_INPUT\_PORTS.

10. ADM issues the run matrix and run COPP commands and monitors the acknowledgement for proper connections. All POPP initializations are made per the user settings, and data is passed between the mic, Hexagon, and the apps processor for recording and stored. This data processing is sent using ASM\_DATA\_CMD\_READ (0x10BDA), and the acknowledgement is issued using ASM\_DATA\_EVENT\_READ\_DONE (0x10BE0).

```

45     00:11:04.082      ./src/AudioStreamMgr_AprIf.cpp 00085 AudioStreamMgr:
46     Rec cmd 0x10bda at [Addr=0x407, Port=0x101]
47     00:11:04.083      ../../../../datalink/aprtal/src/aprtalsmd.c 00300
48     apr_tal_smd_rx_cb: bytes_remaining 32, size 0
49     00:11:04.202      ./src/wdt.c 00173 WDT kick at 441 every 200 ms

```

```

1      00:11:04.205      ./src/AudioStreamMgr_AprIf.cpp  00299  AudioStreamMgr:
2      ISR status 0x0, ASM_DATA_EVENT_READ_DONE [0xc067a990,36]
3      00:11:04.205      ../../../../core/src/aprv2_drv.c  00604
4      alloc(00000407,C04993C8)
5      00:11:04.205      ../../../../core/src/aprv2_drv.c  00557
6      send(00000407,C04993C8)
7      00:11:04.205      ../../../../core/src/aprv2_drv.c  00706
8      free(00000407,C0498DB0)
9      00:11:04.205      ../../../../datalink/aprtal/src/aprtalsmd.c  00220  src
10     <0407,0101>,dst <0507,0101> Token 00000001 Opcode 00010BE0

```

11. When the user hits STOP in the user interface, first the 0x10305 (ADM\_CMD\_COPP\_CLOSE) is issued, then the AFE is stopped and ports are released. Command 0x100cb is AFE\_PORT\_CMD\_STOP.

```

16     00:11:23.986      ./src/AudDevMgr_AprIf.cpp  00084  At 0x4080000 APRCMD
17     0x10305 ADM rcvd payload size 0
18     00:11:23.986      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
19     apr_tal_smd_rx_cb: bytes_remaining 20, size 0
20     00:11:23.986      ./src/AudDevMgr.cpp  00241  ADM: rcvd APR msg [opcode,
21     dst port] = [0x10305, 0x0]
22     00:11:23.986      ./src/AudDevMgr.cpp  00573  ADM processing
23     ADM_CMD_COPP_CLOSE, COPP ID = 0
24     00:11:23.987      ./src/AudDevMgr.cpp  00670  As part of Rx
25     ADM_CMD_COPP_CLOSE, issuing DISABLE_MT_MX_OUT_PORT cmd
26     00:11:23.987      ./src/MixerSvc.cpp  00337  MtMx #0: rcvd custom msg
27     0x180002
29     00:11:24.033      ./src/AFESvcAprIf.cpp  00083  At <0x404> APRCMD <0x
30     100cb> AfeSvc: Rec cmd.
31     00:11:24.033      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
32     apr_tal_smd_rx_cb: bytes_remaining 24, size 0
33     00:11:24.033      ./AFEDalDeviceDriver/src/AFEDalDeviceDriver.cpp  03956
34     Audio DMA Disable
35     00:11:24.033      src/AFEVfrDriver.cpp  00092  removed port
36     port_id=0,dir=0 from vfr sync
37     00:11:24.033      ./src/AFEPortAprHandler.cpp  00257  AFECmdDmaSTOP: Dma
38     Stop Success: dir=0, intf=0.

```

12. ASM\_STREAM\_CMD\_STOP (0x10BCD) stops the ASM from receiving or sending any new streams.

```

43     00:11:24.041      ./src/AudioStreamMgr_AprIf.cpp  00085  AudioStreamMgr:
44     Rec cmd 0x10bcd at [Addr=0x407, Port=0x101]
45     00:11:24.041      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
46     apr_tal_smd_rx_cb: bytes_remaining 20, size 0
47     00:11:24.041      ./src/AudioStreamMgr.cpp  00342  Signal received = 4 !
48     00:11:24.041      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00101
49     AudioStreamMgr [0,1]: receive apr command 0x10bcd at port [0x 101]

```

```

1      00:11:24.041      ./src/AudioStreamMgr_Session.cpp 00699 AudioStreamMgr
2      [0,1]: disconnecting Svc [0x150000,0xb04c] to Svc [0xe0000,0xb04b]!
3      00:11:24.041      ./src/AudioStreamMgr_Session.cpp 00699 AudioStreamMgr
4      [0,1]: disconnecting Svc [0xe0000,0xb04b] to Svc [0xf0000,0x0]!
5      00:11:24.041      ./src/AudioDecSvc.cpp 00481 Message ID 0x      4!!
6      00:11:24.041      ./src/AudioDecSvc.cpp 01515 Decoder Service is
7      disconnecting from downstream service...
8      00:11:24.041      ./src/audproc_svc.cpp 00993 PAA40 audproc_svc:
9      Instance 0xC04ED570 disconnecting handle 0xC050B9F0
10     00:11:24.042      ./src/AudioDecSvc.cpp 01530 Disconnection Succeeded.

```

11

12 13. The command for disconnecting COPP, POPP, and matrix is issued, followed by destroying  
13 COPP and POPP and releasing any hold buffers.

## 14 5.2.1 Hexagon messages for Audio Playback

15 See Section 3.1 of [Q4] for the complete sequence flow.

16 1. Hexagon receives the following command:

17

```

18     00:11:15. 413      . /src/AudioStreamMgr_AprIf.cpp 00085
19     AudioStreamMgr: Rec cmd 0x10bca at [Addr=0x407, Port=0x101]

```

20

21 As seen in [Q4], this command refers to ASM\_STREAM\_CMD\_OPEN\_WRITE. This  
22 command is used to create an audio stream for playback.

23 2. The ASM creates the decoder (in this example, it is Linear PCM, 0x10BE5 input format) and  
24 postprocessing services.

25

```

26     00:11:15.415      ./src/AudioStreamMgr_Util.cpp 00220 AudioStreamMgr
27     [0,1]: Input Format 0x10be5
28     00:11:15.415      ./src/AudioStreamMgr_Util.cpp 00567 AudioStreamMgr
29     [0,1]: Creating Decoder Service
30     00:11:15.415      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc066bb78
31     NumItems=16
32     00:11:15.415      ./src/wdt.c 00255 Exiting Power Collapse
33     00:11:15.415      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc066b720
34     NumItems=8
35     00:11:15.415      ./src/Hexos_Queue.cpp 00096 Q CREATE: Q=0xc066b900
36     NumItems=2
37     00:11:15.415      ./src/Hexos_Thread.cpp 00150 THRD CREATE:
38     Thread=0xb04c Name(Hex)= 41, 44, 65, 63, 0, 0, 0, 0
39     00:11:15.415      ./src/AudioStreamMgr_Util.cpp 00594 AudioStreamMgr
40     [0,1]: Decoder Service [0x150000,0xb04c] Created
41     00:11:15.415      ./src/AudioStreamMgr_Util.cpp 00701 AudioStreamMgr
42     [0,1]: Creating Post-Proc Service
43     00:11:15.415      ./src/audproc_svc.cpp 00196 audproc_svc: Creating
44     00:11:15.415      ./src/audproc_paramhandler.cpp 00344 P0 audproc_svc:
45     Set output media Begin: keepInputFreq: 0, outputFreq: 48000,
46     useNativeNumChannels: 1, outputChannels: 0

```

### 3. Initialization of the POPP is issued:

```

00:11:15.416      ./src/audproc_appi_init.cpp 00848 PAA40 audproc_svc:
Initializing POPP Begin
00:11:15.416      ./src/appi_combopp.cpp 00095 APPI ComboPP Getsize
done, requires 115840 bytes
00:11:15.416      ./src/appi_genericresampler.cpp 00117 Generic
Resampler APPI Init
00:11:15.416      ./src/appi_softvolumecontrols.cpp 00099 Soft Volume
Controls APPI Init
00:11:15.416      ./src/appi_softvolumecontrols_utils.cpp 00041
audproc_svc: SVC left channel gain setting in linear scale 8192
00:11:15.416      ./src/appi_softvolumecontrols_utils.cpp 00042
audproc_svc: SVC right channel gain setting in linear scale 8192
00:11:15.416      ./src/appi_softvolumecontrols_utils.cpp 00060
audproc_svc: SVC master gain setting in linear scale 8192
00:11:15.417      ./src/appi_combopp.cpp 00215 APPI ComboPP Init done
with outformat 2,16,48000,1,1
00:11:15.417      ./src/audproc_appi_spa.cpp 00121 SPA APPI Init
00:11:15.417      ./src/audproc_appi_init.cpp 01010 PAA40 audproc_svc:
Initializing POPP End

```

### 4. The matrix connection command to connect POPP services to matrix services is issued.

```

00:11:15.418      ./src/AudDevMgr.cpp 00509 As part of Rx Stream
Connect, issuing CONNECT_MT_MX_IN_PORT cmd

```

### 5. The created decoder service is connected with the postprocessing service:

```

00:11:15.420      ./src/AudioDecSvc.cpp 01481 Decoder service
connecting to Post-Proc service

```

### 6. The memory mapping command (0x10C45) is issued.

```

00:11:15.421      ./src/AudioStreamMgr_AprIf.cpp 00085 AudioStreamMgr:
Rec cmd 0x10c45 at [Addr=0x407, Port=0x0]

```

### 7. The command to configure AFE, AFE\_PORT\_AUDIO\_IF\_CONFIG, (0x100D3), is issued.

This command configures the port parameters.

```

00:11:15.467      ./src/AFESvcAprIf.cpp 00083 At <0x404> APRCMD <0x
100d3> AfeSvc: Rec cmd.

```



8. AFE\_PORT\_CMD\_START (0x100CA) is issued to start the DMAs on the specified hardware port, and acknowledgement is received.

```
00:11:15.468      ./src/AFESvcAprIf.cpp  00083  At <0x404> APRCMD <0x
100ca> AfeSvc: Rec cmd.
...
00:11:15.475      ./src/AFEPortAprHandler.cpp  00113  AFECmdDmaStart: Dma
Start Success: dir=0, intf=0.
```

9. After the AFE and DMA are ready, COPP services are created.

```
00:11:15.522      ./src/AudDevMgr.cpp  00241  ADM: rcvd APR msg [opcode,
dst port] = [0x10304, 0x0]
00:11:15.522      ./src/AudDevMgr.cpp  01088  ADM processing
ADM_CMD_COPP_OPEN
00:11:15.522      ./src/AudDevMgr.cpp  01323  As part of Rx
ADM_CMD_COPP_OPEN, issuing CREATE_COPP cmd
00:11:15.522      ./src/audproc_svc.cpp  00196  audproc_svc: Creating
00:11:15.522      ./src/audproc_paramhandler.cpp  00344  P0 audproc_svc:
Set output media Begin: keepInputFreq: 1, outputFreq: 0,
useNativeNumChannels: 1, outputChannels: 0
```

10. COPP commands to all its modules, such as volume control, etc., are initialized.

```
00:11:15.523      ./src/audproc_appi_init.cpp  00199  PCC42 audproc_svc:
Initializing copp Begin
...
00:11:15.526      ./src/audproc_appi_init.cpp  00473  PCC42 audproc_svc:
Initializing copp End
```

11. The COPP and AFE connection command is issued.

```
00:11:15.526      ./src/AudDevMgr.cpp  01372  As part of Rx
ADM_CMD_COPP_OPEN, issuing AFE_CONNECT_REQ cmd
00:11:15.527      ./src/AudDevMgr.cpp  01380  As part of Rx
ADM_CMD_COPP_OPEN, issuing CONNECT_COPP cmd
```

12. After successful connection between COPP and AFE, the connection between the COPP and the matrix is issued.

```
00:11:15.527      ./src/AudDevMgr.cpp  01393  As part of Rx
ADM_CMD_COPP_OPEN, issuing CONNECT_MT_MX_OUT_PORT cmd
00:11:15.528      ./src/AudDevMgr.cpp  01411  As part of Rx
ADM_CMD_COPP_OPEN, issuing RUN_MT_MX_OUTPUT_PORT cmd
```

```

1      00:11:15.528      ./src/AudDevMgr.cpp  01488  As part of
2      ADM_CMD_COPP_OPEN, issuing RUN_COPP cmd
3      00:11:15.528      ./src/audproc_svc.cpp  01207  PCC42 audproc_svc: Run
4      Begin
5      00:11:15.528      ./src/audproc_svc.cpp  01224  PCC42 audproc_svc: Run
6      End, sent ack

```

13. ADM registers the callbacks of COPP service.

```

10     00:11:15.528      ./src/AudDevMgr_DataBase.cpp  00064  ADM: Registered
11     PP's CB

```

14. After the initialization of the COPP modules, such as IIR filters, volume control, etc., is completed, updates to the matrix mapping between session IDs and COPP IDs are provided.

```

16     00:11:15.530      ./src/AudDevMgr.cpp  00737  ADM processing
17     ADM_CMD_MATRIX_MAP_ROUTINGS, matrix ID = 0, num sessions = 1

```

15. The COPP parameters are set per the requirement of the device.

```

21     00:11:15.533      ./src/AudDevMgr.cpp  00241  ADM: rcvd APR msg [opcode,
22     dst port] = [0x10306, 0x0]
23     00:11:15.535      ./src/AudDevMgr.cpp  00851  ADM processing
24     ADM_CMD_SET_PARAMS

```

16. The data is transferred between the apps processor and Hexagon, and every time the data is written into ASM buffers, ASM\_DATA\_CMD\_WRITE (0x10BD9) is issued and is acknowledged by Hexagon using ASM\_DATA\_EVENT\_WRITE\_DONE (0x10BDF), also saying that the buffer is empty again.

```

31     00:11:15.428      ./src/AudioStreamMgr_AprIf.cpp  00085  AudioStreamMgr:
32     Rec cmd 0x10bd9 at [Addr=0x407, Port=0x101]
33     00:11:15.430      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
34     apr_tal_smd_rx_cb: bytes_remaining 44, size 0
35     00:11:15.430      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
36     apr_tal_smd_rx_cb: bytes_remaining 0, size 4096
37     00:11:15.430      ./src/AudioStreamMgr_AprIf.cpp  00264  AudioStreamMgr:
38     ISR status 0x0, ASM_DATA_EVENT_WRITE_DONE 0x 10bdf , [0x0,0]

```

## 5.2.2 Device switch sequence

The device switch is carried out by AFE and ADM. The following sequence of steps happens before the actual device switch can happen.

1. The AFE\_PORT\_CMD\_STOP (0x100CB) command is issued to stop the port.

```
00:50:13.255      ./src/AFESvcAprIf.cpp 00083 At <0x404> APRCMD <0x
100cb> AfeSvc: Rec cmd.
```

2. With the AFE stopped, the DMA must be stopped, as DMA is attached to AFE.

```
00:50:13.255      ./AFEDalDeviceDriver/src/AFEDalDeviceDriver.cpp 03956
Audio DMA Disable 00:50:13.255 src/AFEVfrDriver.cpp 00092 removed port
port_id=0,dir=0 from vfr sync
00:50:13.255      ./src/AFEPortAprHandler.cpp 00257 AFECmdDmaSTOP: Dma
Stop Success: dir=0, intf=0. 00:50:13.256      ./src/AFEMmpm.cpp 00151
MMPM_Request for AFE MIPS success (sleep)
00:50:13.260      ./src/AFEMmpm.cpp 00172 MMPM_Request for AFE BW
success (sleep)
```

3. The ADM\_CMD\_COPP\_CLOSE (0x10305) command is issued to disconnect the COPP from AFE ports. As part of the ADM\_CMD\_COPP\_CLOSE, the DISABLE\_MT\_MX\_OUT\_PORT command to disable the matrix is issued.

```
00:50:13.263      ./src/AudDevMgr_AprIf.cpp 00084 At 0x4080000 APRCMD
0x10305 ADM rcvd payload size 0
00:50:13.263      ../../../../datalink/aprtal/src/aprtalsmd.c 00300
apr_tal_smd_rx_cb: bytes_remaining 20, size 0
00:50:13.263      ./src/AudDevMgr.cpp 00241 ADM: rcvd APR msg [opcode,
dst port] = [0x10305, 0x0]
00:50:13.263      ./src/AudDevMgr.cpp 00573 ADM processing
ADM_CMD_COPP_CLOSE, COPP ID = 0
00:50:13.263      ./src/AudDevMgr.cpp 00670 As part of Rx
ADM_CMD_COPP_CLOSE, issuing DISABLE_MT_MX_OUT_PORT cmd
```

4. The COPP\_DISCONNECT command, followed by destroying the COPP instances, is issued as part of ADM\_CMD\_COPP\_CLOSE.

```
00:50:13.263      ./src/AudDevMgr.cpp 00675 As part of Rx
ADM_CMD_COPP_CLOSE, issuing COPP_DISCONNECT cmd
00:50:13.265      ./src/audproc_svc.cpp 00993 PCC16 audproc_svc:
Instance 0xC066EDF0 disconnecting handle 0xC069248C
00:50:13.265      ./src/audproc_svc.cpp 01009 PCC16 audproc_svc:
Disconnection Succeeded.
00:50:13.265      ./src/AudDevMgr.cpp 00679 As part of Rx
ADM_CMD_COPP_CLOSE, issuing AFE_PORT_DISCONNECT cmd
```

```

1      00:50:13.265      ./src/AudDevMgr.cpp  00685  As part of Rx
2      ADM_CMD_COPP_CLOSE, issuing COPP_DESTROY cmd
3      00:50:13.265      ./src/audproc_svc.cpp  00655  PCC16 audproc_svc:
4      Destroy yourself Begin
5      00:50:13.265      ./src/audproc_svc.cpp  00394  PCC16 audproc_svc:
6      Destroy Svc Begin

```

Any held buffers are also returned to the client. In destroying the COPP, modules within a COPP topology chain, such as volume control, any IIR, FIR filters, and device calibration filters, are also destroyed.

```

12     00:50:13.265      ./src/audproc_appi_topo.cpp  00577  PCC16 audproc_svc:
13     GenTopo Destroy Begin.
14     00:50:13.265      ./src/appi_softvolumecontrols.cpp  00307  Soft Volume
15     Controls APPI End
16     00:50:13.265      ./src/appi_combopp.cpp  00475  APPI ComboPP End done
17     00:50:13.265      ./src/appi_pbe.cpp  00373  APPI PBE End done
18     00:50:13.265      ./src/appi_multibandirr.cpp  00313  MultiBandIIR APPI
19     End
20     00:50:13.265      ./src/appi_mbdrc.cpp  00378  MBDRC APPI End
21     00:50:13.265      ./src/appi_qensemble.cpp  00234  QEnsemble APPI end
22     00:50:13.265      ./src/audproc_appi_spa.cpp  00274  SPA APPI End
23     00:50:13.265      ./src/audproc_appi_topo.cpp  00597  PCC16 audproc_svc:
24     GenTopo Destroy End, now freeing GenTopo.
25     00:50:13.265      ./src/audproc_svc.cpp  00425  audproc_svc: Destroy Svc
26     End
27     00:50:13.266      ./src/audproc_svc.cpp  00675  audproc_svc: Destroy
28     yourself End, sent ack
29     00:50:13.266      ./src/AudDevMgr.cpp  00689  As part of Rx
30     ADM_CMD_COPP_CLOSE, issuing DISCONNECT_MT_MX_OUT_PORT cmd

```

5. Any callbacks to COPP modules are disabled.

```

34     00:50:13.266      ./src/AudDevMgr.cpp  00693  All cmds to stop Rx Device
35     have been issued
36     00:50:13.266      ./src/AudDevMgr.cpp  00701  ADM: Waiting for COPP to
37     join
38     00:50:13.266      ./src/AudDevMgr_DataBase.cpp  00078  ADM: De-registered
39     PP's CB

```

**NOTE:** After all the individual modules and topology associated with COPP are destroyed and after disconnection with matrix and AFE, all the previous device-associated processing is cleared. Next, some power-management bookkeeping takes place.

6. For a device switch to happen, a new device-associated calibration takes place. This process starts with the command AFE\_PORT\_AUDIO\_IF\_CONFIG (0x100D3), operation code for the AFE service to configure the audio interface.

```
00:50:13.310      ./src/AFESvcAprIf.cpp 00083 At <0x404> APRCMD <0x
100d3> AfeSvc: Rec cmd.
```

7. The AFE\_PORT\_CMD\_START (0x100CA) command is issued to start a new AFE port. This is followed by the DMA start command.

```
00:50:13.312      ./src/AFESvcAprIf.cpp 00083 At <0x404> APRCMD <0x
100ca> AfeSvc: Rec cmd. ... 00:50:13.318      ./src/AFEPortAprHandler.cpp
00113 AFECmdDmaStart: Dma Start Success: dir=0, intf=0.
```

8. ADM\_CMD\_COPP\_OPEN (0x10304) is issued.

```
00:50:13.365      ./src/AudDevMgr_AprIf.cpp 00084 At 0x4080000 APRCMD
0x10304 ADM rcvd payload size 20
00:50:13.365      ../../../../datalink/aprtal/src/aprtalsmd.c 00300
apr_tal_smd_rx_cb: bytes_remaining 40, size 0
00:50:13.365      ./src/AudDevMgr.cpp 00241 ADM: rcvd APR msg [opcode,
dst port] = [0x10304, 0x0]
00:50:13.365      ./src/AudDevMgr.cpp 01088 ADM processing
ADM_CMD_COPP_OPEN
00:50:13.365      ./src/AudDevMgr.cpp 01323 As part of Rx
ADM_CMD_COPP_OPEN, issuing CREATE_COPP cmd
00:50:13.365      ./src/audproc_svc.cpp 00196 audproc_svc: Creating
```

9. Any input, output media-based calibrations are set:

```
00:50:13.365      ./src/audproc_paramhandler.cpp 00344 P0 audproc_svc:
Set output media Begin: keepInputFreq: 1, outputFreq: 0,
useNativeNumChannels: 1, outputChannels: 0
00:50:13.365      ./src/audproc_paramhandler.cpp 00396 audproc_svc:
Setting output media End
```

10. Individual modules in the COPP topology chain, such as volume control, and any device-calibration filter modules are initiated and associated threads are created.

```
00:50:13.365      ./src/appi_pbe.cpp 00089 APPI PBE Getsize done,
requires 25288 bytes
00:50:13.365      ./src/audproc_appi_init.cpp 00300 audproc_svc:
Requesting 229384 bytes of memory for COPP!
00:50:13.365      ./src/appi_softvolumecontrols.cpp 00099 Soft Volume
Controls APPI Init
00:50:13.365      ./src/appi_softvolumecontrols_utils.cpp 00041
audproc_svc: SVC left channel gain setting in linear scale 8192
00:50:13.365      ./src/appi_softvolumecontrols_utils.cpp 00042
```

```

1      audproc_svc: SVC right channel gain setting in linear scale 8192
2      00:50:13.365      ./src/appi_softvolumecontrols_utils.cpp 00060
3      audproc_svc: SVC master gain setting in linear scale 8192
4      00:50:13.366      ./src/appi_combopp.cpp 00215 APPI ComboPP Init done
5      with outformat 2,16,48000,1,1
6      00:50:13.366      ./src/appi_pbe.cpp 00226 APPI PBE Init done with
7      outformat 2,16,48000,1,1
8      00:50:13.366      ./src/appi_multibandir.cpp 00116 MultiBandIIR APPI
9      Init for 5 bands
10     00:50:13.366      ./src/appi_mbdrc.cpp 00121 MBDRC APPI Init
11     00:50:13.366      ./src/appi_qensemble.cpp 00095 QEnsemble APPI init
12     00:50:13.366      ./src/audproc_app_i_spa.cpp 00121 SPA APPI Init
13     00:50:13.367      ./src/audproc_app_i_init.cpp 00473 PCC18 audproc_svc:
14     Initializing copp End
15     00:50:13.367      ./src/Hexos_Thread.cpp 00150 THRD CREATE:
16     Thread=0x504b Name(Hex)= 43, 43, 31, 38, 0, 0, 0, 0
17     00:50:13.367      ./src/audproc_svc.cpp 00498 PCC18 audproc_svc:
18     Entering workloop with fields Buf=60000000, Data=c0000000,
19     Command=40000000, pauseTimer=10000000
20
21     11. ADM issues a command to connect to the AFE.
22
23     00:50:13.367      ./src/AudDevMgr.cpp 01372 As part of Rx
24     ADM_CMD_COPP_OPEN, issuing AFE_CONNECT_REQ cmd
25
26     12. ADM issues a command to connect COPP, followed by the matrix connection.
27
28     00:50:13.368      ./src/AudDevMgr.cpp 01380 As part of Rx
29     ADM_CMD_COPP_OPEN, issuing CONNECT_COPP cmd
30     00:50:13.368      ./src/audproc_svc.cpp 00834 PCC18 audproc_svc:
31     instance 0xC066EDF0 connecting to SvcID 0x f0000
32     00:50:13.368      ./src/AFEPortManager.cpp 01204 AfePort intf=0, dir=0:
33     Audio data not available in Rx path from client 0. CurrTime-PrevTime =
34     49146 us. Num Underrun = 1!!
35     00:50:13.368      ./src/audproc_svc.cpp 00880 PCC18 audproc_svc: COPP
36     Connection Succeeded.
37     00:50:13.368      ./src/AudDevMgr.cpp 01393 As part of Rx
38     ADM_CMD_COPP_OPEN, issuing CONNECT_MT_MX_OUT_PORT cmd
39
40     13. Matrix and COPP run commands are issued as part of ADM_CMD_COPP_OPEN.
41
42     00:50:13.370      ./src/AudDevMgr.cpp 01411 As part of Rx
43     ADM_CMD_COPP_OPEN, issuing RUN_MT_MX_OUTPUT_PORT cmd
44     00:50:13.370      ./src/MixerSvc.cpp 00337 MtMx #0: rcvd custom msg
45     0x180004
46     00:50:13.370      ./src/MixerSvc_MsgHandlers.cpp 00564 MtMx #0:
47     Processing cmd 0x180004 [port ID, port dir, start flag, start time] = [0,
48     2, 0x0, 8668712731819376640]

```

```

1      00:50:13.370      ./src/MixerSvc_MsgHandlers.cpp  00938  MtMx #0: Leaving
2      cmd [0x180004] handler with status 0x0
3      00:50:13.370      ./src/AudDevMgr.cpp  01488  As part of
4      ADM_CMD_COPP_OPEN, issuing RUN_COPP cmd
5      00:50:13.370      ./src/audproc_svc.cpp  01207  PCC18 audproc_svc: Run
6      Begin
7      00:50:13.370      ./src/audproc_svc.cpp  01224  PCC18 audproc_svc: Run
8      End, sent ack
9

```

14. The COPP callback functions are registered.

```

12     00:50:13.370      ./src/AudDevMgr_DataBase.cpp  00064  ADM: Registered
13     PP's CB
14

```

15. After all the connections are successful, the ADM\_CMDRSP\_COPP\_OPEN (0x1030A) command for the status and COPP ID for the ADM\_CMD\_COPP\_OPEN command are returned. All individual COPP modules, such as volume control, etc., are reinitiated with the device calibration requirements.

```

20     00:50:13.370      ./src/appi_softvolumecontrols.cpp  00182  Soft Volume
21     Controls APPI Reinit
22     00:50:13.370      ../../../../datalink/aprtal/src/aprtalsmd.c  00220  src
23     <0408,0000>,dst <0508,0000> Token 00000000 Opcode 0001030A
24     00:50:13.370      ../../../../datalink/aprtal/src/aprtalsmd.c  00232
25     apr_tal_smd_tx_cb: Size = 32
26     00:50:13.370      ./src/appi_combopp.cpp  00269  APPI ComboPP Reinit done
27     with outformat 2,16,48000,1,0
28     00:50:13.370      ../../../../core/src/aprv2_drv.c  00706
29     free(00000000,C049B448)
30     00:50:13.370      ./src/appi_pbe.cpp  00267  APPI PBE Reinit done with
31     outformat 2,16,48000,1,0
32     00:50:13.370      ../../../../datalink/aprtal/src/aprtalsmd.c  00232
33     apr_tal_smd_tx_cb: Size = 0
34     00:50:13.370      ./src/appi_multibandir.cpp  00227  MultiBandIIR APPI
35     Reinit
36     00:50:13.370      ./src/appi_mbdrc.cpp  00271  MBDRC APPI Reinit
37     00:50:13.371      ./src/appi_qensemble.cpp  00134  QEnsemble APPI reinit
38

```

16. The ADM\_CMD\_MATRIX\_MAP\_ROUTINGS (0x010301) command is issued to update the matrix mapping between session IDs and COPP IDs.

```

42     00:50:13.371      ./src/AudDevMgr.cpp  00241  ADM: rcvd APR msg [opcode,
43     dst port] = [0x10301, 0x0]
44     00:50:13.371      ./src/audproc_msghandler.cpp  00112  PCC18 audproc_svc:
45     Parse Media type End
46     00:50:13.371      ./src/AudDevMgr.cpp  00737  ADM processing
47     ADM_CMD_MATRIX_MAP_ROUTINGS, matrix ID = 0, num sessions = 1
48

```

This is followed by ADM\_CMD\_SET\_PP\_PARAMS (0x10306) to set one or more parameters in a COPP.

```

00:50:13.375      ./src/AudDevMgr_AprIf.cpp  00084  At 0x4080000 APRCMD
0x10306 ADM rcvd payload size 28
00:50:13.375      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
aprtal_smd_rx_cb: bytes_remaining 48, size 0
00:50:13.375      ./src/AudDevMgr.cpp  00241  ADM: rcvd APR msg [opcode,
dst port] = [0x10306, 0x0]
00:50:13.375      ./src/AudDevMgr.cpp  00851  ADM processing
ADM_CMD_SET_PARAMS
00:50:13.375      ./src/audproc_svc.cpp  01057  PCC18 audproc_svc: Set
ParamId 0x1, Client token=0x0
00:50:13.375      ./src/audproc_msghandler.cpp  00925  PCC18 audproc_svc:
Set Param Outband Begin
00:50:13.375      ./src/audproc_appi_topo.cpp  00437  PCC18 audproc_svc:
GenTopo Set params Begin.
00:50:13.375      ./src/appi_combopp.cpp  00559  APPI ComboPP QCPR Set
Enable/Disable, 0
00:50:13.375      ./src/appi_combopp.cpp  00662  APPI ComboPP Set param
done ... 00:50:13.377      ./src/audproc_appi_topo.cpp  00493  PCC18
audproc_svc: GenTopo Set params End.
00:50:13.377      ./src/audproc_msghandler.cpp  00968  PCC18 audproc_svc:
Set Param Outband End, sent ack

```

**NOTE:** Between each command issued, using the APR layers and acknowledgement is always required.

```

00:50:13.381      ../../../../datalink/aprtal/src/aprtalsmd.c  00220  src
<0408,0000>,dst <0508,0000> Token 00000000 Opcode 000110E8

```

17. The buffers are rendered and audio is played again.

```

00:50:13.395      ./src/AudioStreamMgr_AprIf.cpp  00085  AudioStreamMgr:
Rec cmd 0x10bd9 at [Addr=0x407, Port=0x101]
00:50:13.395      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
aprtal_smd_rx_cb: bytes_remaining 44, size 0
00:50:13.425      ./src/AudioStreamMgr_AprIf.cpp  00264  AudioStreamMgr:
ISR status 0x0, ASM_DATA_EVENT_WRITE_DONE 0x 10bdf , [0x0,0]

```



## 5.2.3 Fast forward/rewind sequence during playback

This section describes the commands received and log messages printed during fast forward and rewind of audio.

1. Just before the issue of ASM\_SESSION\_CMD\_PAUSE, Hexagon received a message:

```
00:07:11.866      ./src/AudioStreamMgr_AprIf.cpp  00085  AudioStreamMgr:
Rec cmd 0x10bd9 at [Addr=0x407, Port=0x101]
```

2. Then Hexagon receives an ASM\_SESSION\_CMD\_PAUSE (0x10BD3) command.

```
00:07:13.140      ./src/AudioStreamMgr_AprIf.cpp  00085  AudioStreamMgr:
Rec cmd 0x10bd3 at [Addr=0x407, Port=0x101]
00:07:13.140      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
apr_tal_smd_rx_cb: bytes_remaining 20, size 0
00:07:13.140      ./src/AudioStreamMgr.cpp  00342  Signal received = 4 !
00:07:13.140      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00101
AudioStreamMgr [0,1]: receive apr command 0x10bd3 at port [0x 101]
00:07:13.140      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01197
AudioStreamMgr [0,1]: enter PAUSE command handler
```

3. The POPP services are also paused by the Hexagon processor.

```
00:07:13.140      ./src/AudioStreamMgr_Session.cpp  00801  AudioStreamMgr
[0,1]: Pausing PP svc 0x10100
00:07:13.140      ./src/audproc_svc.cpp  01148  PAA2F audproc_svc: Pause
Handler Begin
00:07:13.140      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01242
AudioStreamMgr: leave PAUSE command handler0
00:07:13.140      ./src/audproc_svc.cpp  01183  PAA2F audproc_svc: Pause
Handler End, sent ack 00:07:13.140      ./src/AudioStreamMgr_Session.cpp
00801  AudioStreamMgr [0,1]: Pausing PP svc 0x10100
00:07:13.140      ./src/audproc_svc.cpp  01148  PAA2F audproc_svc: Pause
Handler Begin
00:07:13.140      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01242
AudioStreamMgr: leave PAUSE command handler0
00:07:13.140      ./src/audproc_svc.cpp  01183  PAA2F audproc_svc: Pause
Handler End, sent ack
00:07:13.141      ./src/AudioStreamMgr.cpp  00342  Signal received = 8 !
00:07:13.141      ./src/AudioStreamMgr_SessionRespHandler.cpp  01152
AudioStreamMgr [0,1]: Pause ACK Tok 0x10100, Res 0x0
00:07:13.141      ./src/AudioStreamMgr_DevIF.cpp  00319  AudioStreamMgr
[0,1]: Pausing Matrix svc 0x10201
```

4. After the POPP services are paused, the acknowledgement is sent to the apps processor.

```
00:07:13.141      ./src/AudioStreamMgr_SessionRespHandler.cpp  01152
AudioStreamMgr [0,1]: Pause ACK Tok 0x10201, Res 0x0
```

```

1      00:07:13.141      ./src/AudioStreamMgr_AprIf.cpp  00357  AudioStreamMgr:
2      ISR status 0x0, Basic Ack: 0x10bd3 , [0x0,0]
3

```

5. The ASM\_SESSION\_CMD\_GET\_SESSION\_TIME (0x10BD4) and  
 ASM\_SESSION\_CMDRSP\_GET\_SESSION\_TIME (0x10BD8) commands are exchanged  
 to know where to run when play is resumed.

```

8      00:07:18.262      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00101
9      AudioStreamMgr [0,1]: receive apr command 0x10bd4 at port [0x 101]
10     00:07:18.262      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01047
11     AudioStreamMgr [0,1]: enter Get session time command handler
12     00:07:18.262      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01083
13     AudioStreamMgr [0,1]: Requesting session time to Mixer
14     00:07:18.262      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01098
15     AudioStreamMgr [0,1]: leave get session time handler to mixer
16     00:07:18.262      ./src/MixerSvc.cpp  00337  MtMx #0: rcvd custom msg
17     0x180007
18     00:07:18.262      ./src/MixerSvc_MsgHandlers.cpp  01720  MtMx #0:
19     Processing cmd 0x180007 [port ID, port dir] = [0, 1]
20     00:07:18.262      ./src/MixerSvc_MsgHandlers.cpp  01734  MtMx #0 cmd
21     [0x180007] handler:[i/p port id, session time msw, session time lsw] = [0,
22     0x0, 0x9980e0]
23     00:07:18.262      ./src/AudioStreamMgr.cpp  00342  Signal received = 8 !
24     00:07:18.262      ./src/MixerSvc_MsgHandlers.cpp  01745  MtMx #0: Leaving
25     cmd [0x180007] handler with status 0x0
26     00:07:18.262      ./src/AudioStreamMgr_SessionRespHandler.cpp  00884
27     AudioStreamMgr [0,1]: Get Session Time command ACK
28     00:07:18.262      ./src/AudioStreamMgr_AprIf.cpp  00341  AudioStreamMgr:
29     ISR status 0x0, Ack 0x10bd8 , [0xc060a9d0,12]
30

```

6. The ASM\_STREAM\_CMD\_FLUSH (0x10BCE) command is issued, and for any  
 ASM\_DATA\_CMD\_WRITE command, if it was received previously, its related  
 ASM\_DATA\_EVENT\_WRITE\_DONE is issued.

```

35     00:07:18.265      ./src/AudioStreamMgr_AprIf.cpp  00085  AudioStreamMgr:
36     Rec cmd 0x10bce at [Addr=0x407, Port=0x101]
37     00:07:18.265      ../../../../datalink/aprtal/src/aprtalsmd.c  00300
38     apr_tal_smd_rx_cb: bytes_remaining 20, size 0
39     00:07:18.265      ./src/AudioStreamMgr.cpp  00342  Signal received = 4 !
40     00:07:18.265      ./src/AudioStreamMgr_SessionCmdHandler.cpp  00101
41     AudioStreamMgr [0,1]: receive apr command 0x10bce at port [0x 101]
42     00:07:18.265      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01265
43     AudioStreamMgr [0,1]: enter Flush Stream command handler
44     00:07:18.265      ./src/AudioStreamMgr_Session.cpp  00918  AudioStreamMgr
45     [0,1]: Flushing Dec/Enc svc 0x10000
46     00:07:18.265      ./src/AudioDecSvc.cpp  00481  Message ID 0x      7!!
47     00:07:18.265      ./src/AudioStreamMgr_SessionCmdHandler.cpp  01331
48     AudioStreamMgr [0,1]: leave Flush command handler

```

```

1      00:07:18.265      ./src/AudioStreamMgr_AprIf.cpp 00264 AudioStreamMgr:
2      ISR status 0x0, ASM_DATA_EVENT_WRITE_DONE 0x 10bdf , [0x0,0]
3

```

7. After the FLUSH command from the apps processor, Hexagon flushes POPP, matrix services, and other related sessions.

```

7      00:07:18.265      ./src/AudioStreamMgr_Session.cpp 00811 AudioStreamMgr
8      [0,1]: Flushing PP svc 0x10101
9      00:07:18.265      ./src/audproc_svc.cpp 00727 PAA2F audproc_svc: Flush
10     Begin ... 00:07:18.266      ./src/audproc_svc.cpp 00788 PAA2F
11     audproc_svc: Flush End, sent ack
12     00:07:18.266      ./src/AudioStreamMgr_SessionRespHandler.cpp 00975
13     AudioStreamMgr [0,1]: Flush ACK Tok 0x10101, Res 0x0
14     00:07:18.266      ./src/AudioStreamMgr_DevIF.cpp 00322 AudioStreamMgr
15     [0,1]: Flushing Matrix svc 0x10202
16

```

8. The acknowledgement for the FLUSH command is issued from Hexagon.

```

19     00:07:18.267      ./src/AudioStreamMgr_SessionRespHandler.cpp 00975
20     AudioStreamMgr [0,1]: Flush ACK Tok 0x10202, Res 0x0
21     00:07:18.267      ./src/AudioStreamMgr_AprIf.cpp 00357 AudioStreamMgr:
22     ISR status 0x0, Basic Ack: 0x10bce , [0x0,0]
23

```

9. When ASM\_SESSION\_CMD\_RUN (0x10BD2) is received, Hexagon resumes the process.

```

26     00:07:19.401      ./src/AudioStreamMgr_AprIf.cpp 00085 AudioStreamMgr:
27     Rec cmd 0x10bd2 at [Addr=0x407, Port=0x101]
28     00:07:19.402      ../.././datalink/aprtal/src/aprtalsmd.c 00300
29     apr_tal_smd_rx_cb: bytes_remaining 32, size 0
30     00:07:19.402      ./src/AudioStreamMgr.cpp 00342 Signal received = 4 !
31     00:07:19.402      ./src/AudioStreamMgr_SessionCmdHandler.cpp 00101
32     AudioStreamMgr [0,1]: receive apr command 0x10bd2 at port [0x 101]
33     00:07:19.402      ./src/AudioStreamMgr_SessionCmdHandler.cpp 01117
34     AudioStreamMgr [0,1]: enter SessionRun command handler
35     00:07:19.402      ./src/AudioStreamMgr_DevIF.cpp 00360 AudioStreamMgr
36     [0,1]: Running Matrix svc 0x10201
37     00:07:19.402      ./src/AudioStreamMgr_SessionCmdHandler.cpp 01179
38     AudioStreamMgr [0,1]: leave RUN command handler
39

```

**NOTE:** To pause and resume audio playback, only commands ASM\_SESSION\_CMD\_PAUSE and ASM\_SESSION\_CMD\_RUN are sent.

## 5.3 Ulog analysis

### 5.3.1 MPPM log

The ADSP MPPM ulog contains information about MPPM operation; it has the following information:

- Client registration information (client name and ID)
- Client requests (client name, resource requested (MIPS or BW), request parameters)
- Some information regarding clock selection, e.g., aggregated MIPS/BW values, clock plan for ADSP/AHB/MIDI, clock source
- External BW requests
- Voltage level and PMIC mode selection for ADSP voltage rail
- Clock gate status
- PLL voting
- Voting for/against power collapse
- Power collapse entry/exit markers based on calls from Sleep task

This information can be useful for analyzing LPASS behavior in the case of crashes, or for analyzing client requests and clock/voltage selection for power optimizations.

### 5.3.2 Sleep log

The logging system provides the log levels shown in [Table 5-1](#). For each of the log levels, a separate log is created. This allows for the maximum amount of information obtained from the log.

**Table 5-1 Sleep logs**

| Log Name                | Log Size | Per Core? | Description                                                                                                                                                   |
|-------------------------|----------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sleep WarningError.ulog | 256 B    | Yes       | Contains warning and error messages from the sleep subsystem.                                                                                                 |
| Sleep Statistics.ulog   | 512 B    | No        | Contains information about how many times each mode has been entered, and how much time has been spent in each mode.                                          |
| Sleep Requests.ulog     | 1 KB     | No        | Contains information about when various modes get enabled or disabled.                                                                                        |
| Sleep Info.ulog         | 4 KB     | Yes       | Contains information about which modes were or were not chosen.                                                                                               |
| Sleep Profiling.ulog    | 4 KB     | Yes       | Contains information about when the solver and individual modes were entered or exited. Can be used to see how long enter and exit functions take to execute. |
| Sleep Debug.ulog        | 4 KB     | Yes       | Contains verbose information about sleep execution.                                                                                                           |

By default, the logging level is set to Debug. This means that any log message with a level greater than Debug is ignored. This prevents unnecessary logging overhead in the normal use cases. The log level can be modified by calling `sleep_log_set_level()` with the desired log level. This is useful when it is desired to enable more logging for debugging or profiling purposes.

### 5.3.2.1 Sleep warning/error log

This sleep log is called “Sleep WarningError.ulong”, and it contains the following general information:

#### Errors

Whenever the user registers an LPR with the sleep subsystem, error checking will be performed in order to check for validity of the input data. For much of the error checking done at this time, the error is treated as a fatal error, and the system is stopped immediately.

Conditions which occur for which an error message is printed in the sleep log are:

- An LPR is created with an invalid LPR name.
- An LPR is created without an LPRM.
- An LPRM is created with invalid parameters.
- An LPR is registered more than once.
- An LPRM is created with unsupported latency or power tags

#### Warnings

Conditions in which the sleep code may print a warning in the log are:

- When setting the back-off time, the new match value is in the past.
- An LPR is registered with dependencies, but is not pre-synthesized.
- When the sleep code exits, it is past the next scheduled deadline. This is logged as a late wakeup.

This is an example of a late wakeup log message:

```
0x0: WARNING (message: "Late Wakeup") (sleep_end: 0x015D9BCC)
(expected_end: 0x015D9B54) (diff sclk: 120) (diff us: 3662)
```

### 5.3.2.2 Sleep statistics log

This sleep log is called Sleep Statistics.ulong, and it contains the following general information:

- The number of times a mode has been entered
- The amount of time that has been spent in a mode

**NOTE:** This log contains only the latest information for each mode; it does not contain a message for each event that occurs, as in the other logs. Therefore, the timestamp on the messages should not be used for determining when certain modes were entered or exited.

This is an example snippet from the statistics log:

```
0x0: (Mode: px0.shutdown) (Total time in mode: 0x00000000 sclk) (Mode enter
count: 0)
0x0: (Mode: cx0.shutdown) (Total time in mode: 0x00000000 sclk) (Mode enter
count: 0)
0x0: (Mode: cpu_vdd0.power_collapse) (Total time in mode: 0x00000000 sclk)
(Mode enter count: 1391)
0x0: (Mode: cpu_vdd1.power_collapse) (Total time in mode: 0x00000000 sclk)
(Mode enter count: 268)
```

### 5.3.2.3 Sleep requests log

This sleep log is called Sleep Requests.ulong, and it contains the following general information:

- When a mode gets enabled
- When a mode gets disabled
- When a mode gets enabled, but some of its dependencies are not enabled, thus the mode will not get entered

This log is shared between all cores, since all messages logged to this log happen outside of the idle path.

This is an example snippet from the requests log:

```
0x0: Mode enabled (lpr: "cpu_vdd0") (lprm: "power_collapse")
0x0: Synthesized component modes are not enabled(CLpr: 0x00138418) (Core:
0x000000001)
0x0: Mode enabled (lpr: "cpu_vdd1") (lprm: "power_collapse")
0x0: Synthesized component modes are not enabled(CLpr: 0x00138470) (Core:
0x000000002)
0x0: Mode enabled (lpr: "vdd_dig") (lprm: "min_level_0")
0x0: Mode enabled (lpr: "vdd_dig") (lprm: "min_level_1")
```

### 5.3.2.4 Sleep info log

This sleep log is called Sleep Info.ulong, and it contains the following general information:

- Name of the solver
- Solver constraints, i.e., duration, latency budget, etc.
- Name of LPRMs that are chosen for entry
- Reasons for not choosing a certain LPRM for entry

This is an example snippet from the information log:

```
0x0: Solver constraints (hard duration: 0xFFFFFFFF) (soft duration:
0x000000CCC) (latency budget: 0xFFFFFFFF)
0x0: Mode skipped (name: "swfi_core0.clk_gate_with_ramp") (reason:
"Insufficient Power savings") (power: 0x00000CD9 uA*sclk) (pending:
0x00003B93B uA*sclk)
0x0: Mode chosen: ("cpu_vdd0.power_collapse +
swfi_core0.swfi_power_collapse")
0x0: Entering modes
0x0: Exiting modes
```

### 5.3.2.5 Sleep profiling log

This sleep log is called Sleep Profiling.ulong, and it contains the following general information:

- Solver logging
  - Solver entry
  - Solver exit
- LPRM/Sleep logging
  - Sleep thread entry
  - Sleep thread exit
  - Mode entry begin and end
  - Mode exit begin and end

This is an example snippet from the profiling log:

```
0x0: Sleep entry
0x0: Solver Entry (name: "Legacy Solver")
0x0: Solver exit
0x0: Mode entering (lpr: "cpu_vdd0") (lprm: "power_collapse")
0x0: Mode entering (lpr: "swfi_core0") (lprm: "swfi_power_collapse")
0x0: Mode exiting (lpr: "swfi_core0") (lprm: "swfi_power_collapse")
0x0: Mode exiting (lpr: "cpu_vdd0") (lprm: "power_collapse")
0x0: Sleep exit
```

### 5.3.2.6 Sleep debug log

This sleep log is called Sleep Debug.ulong, and it contains the following general information:

- Wake-up reason and associated data
- RPM timed trigger that is sent when entering XO shutdown
- Setting and restoring the backoff time
- Names of LPRs that are registered
- Early exits that are detected, due to pending interrupts.

This is an example snippet from the debug log:

```
0x94754E: Hard wakeup durations (Timer Duration: 0x00000202) (Target
Duration: 0xFFFFFFFF)
0x947559: Setting backoff time (saved match val: 0x00947750) (backoff match
val: 0x00947744) (expected wakeup tick: 0x00947751) (backoff amount:
0x00000000D)
0x947751: Restored match val (restored match val: 0x00947750)
```

### 5.3.2.7 Sleep dump

The sleep dump provides a current snapshot of the state of the sleep subsystem. The cmm script gathers the information by traversing the appropriate list structures in the sleep subsystem. The information that is contained in the dump is:

- LPRs and LPRMs that are registered
- Enter latency, exit latency, and power savings for each LPRM
- Whether each LPRM is enabled or disabled
- List of synthesized LPRMs and their component modes

An example snippet from the sleep dump is:

```
0x0: Core 0
0x0:   Registered LPRs:
0x0:     (Resource Name: cpu_vdd0) (Mode Count: 1)
0x0:       (Mode: cpu_vdd0.power_collapse) (status: Enabled) (enter
latency: 1) (exit latency: 1) (backoff: 1) (Attribute: 0x3)
0x0:     (Resource Name: swfi_core0) (Mode Count: 3)
0x0:       (Mode: swfi_core0.clk_gate_with_ramp) (status: Enabled) (enter
latency: 30) (exit latency: 13) (backoff: 13) (Attribute: 0x3)
0x0:       (Mode: swfi_core0.swfi_power_collapse) (status: Enabled)
(enter latency: 96) (exit latency: 160) (backoff: 59) (Attribute: 0x3)
0x0:       (Mode: swfi_core0.swfi_l2cache_power_collapse) (status:
Enabled) (enter latency: 110) (exit latency: 180) (backoff: 59)
(Attribute: 0x0)
0x0:   Component pre-synthesized modes:
```



```

1      0x0:      (Mode: cpu_vdd0.power_collapse + swfi_core0.swfi_power_collapse)
2      0x0:      (Status: Enabled)
3      0x0:      (Mode: swfi_core0.clk_gate_with_ramp)
4      0x0:      (Status: Enabled)
5      0x0: Core 1
6      0x0:      Registered LPRs:
7      0x0:      (Resource Name: cpu_vdd1) (Mode Count: 1)
8      0x0:      (Mode: cpu_vdd1.power_collapse) (status: Enabled) (enter
9      latency: 2) (exit latency: 2) (backoff: 1) (Attribute: 0x3)
10     0x0:      (Resource Name: swfi_core1) (Mode Count: 3)
11     0x0:      (Mode: swfi_core1.clk_gate_with_ramp) (status: Enabled) (enter
12     latency: 30) (exit latency: 13) (backoff: 13) (Attribute: 0x3)
13     0x0:      (Mode: swfi_core1.swfi_power_collapse) (status: Enabled)
14     (enter latency: 96) (exit latency: 160) (backoff: 59) (Attribute: 0x3)
15     0x0:      (Mode: swfi_core1.swfi_l2cache_power_collapse) (status:
16     Enabled) (enter latency: 110) (exit latency: 180) (backoff: 59)
17     (Attribute: 0x0)
18     0x0:      Component pre-synthesized modes:
19     0x0:      (Mode: cpu_vdd1.power_collapse + swfi_core1.swfi_power_collapse)
20     0x0:      (Status: Enabled)
21     0x0:      (Mode: swfi_core1.clk_gate_with_ramp)
22     0x0:      (Status: Enabled)

```

### 5.3.3 RPMMaster log

The RPM publishes a small log into a very limited area of spare message RAM.

The physical format of the log is the Ulog format used for various other logs.

RPM has a Circular buffer “Raw” log, currently sized at around 4 kB, using a set of IDs and a variable number of parameters per message.

RPM messaging is built around three core concepts:

- **Masters** – These are the subsystems that make requests to the RPM; e.g., on MSM8660, there are three masters, Apps, Modem, and LPASS Hexagon.
- **Resources** – These are the physical components for which the RPM is responsible for controlling, i.e., clocks, voltage rails, etc. Every resource is able to receive independent requests from more than one source.
- **Configuration sets (or “sets”)** – These are a collection of settings (one for each resource) that all fit a logical use case. At the moment, two sets are provided for each master:
  - **Active set** – This set contains all the settings used when the subsystem is awake.
  - **Sleep set** – This set contains all the settings used when the subsystem is sleeping (when an SPM handshake occurs). The active set is also considered the primary set.

Every master has two sets that are stored in the RPM. At any given time one of these sets is operating as the selected set. When a set becomes selected, the requests from that set are sent to the drivers for each respective resource. When the selected set does not have a request for a particular resource, the effective request for that master is inherited from the primary set.

Transitions are made between sets based on triggers. The RPM currently supports two kinds of triggers:

- SPM handshakes
  - Shutdown handshakes automatically trigger a change to the subsystem's sleep set.
  - Bringup handshakes automatically trigger a change to the subsystem's active set.
- Timers
  - A timed trigger can be set up to cause a set change between two sets after being in the selected set for a certain period of time. The most-used timed trigger is a wakeup timer, i.e., a timed trigger in the sleep set that returns to the subsystem's active set after some time has elapsed.
  - Timed triggers from the sleep set always generate a wakeup interrupt.

RPM messages are designed to fulfill this transaction. RPM message content includes:

- Resource to be modified
- Requested setting for resource
- Set for which the request is designated

An example of RPM message looks like this:

```
0x51: rpm_mqueue_put (handle: 0xC0843320) (set: 0) (resource: 9) (sync:
1) (event: 0xC0843388)
0x51: rpm_lprm_disabled
0x51: rpm_package_begin (set: 0) (resource: 9)
0x51: rpm_sending_message
0x5C: rpm_mqueue_remove (set: 0) (resource: 9)
0x5D: rpm_lprm_enabled
```

Sets are as described as:

- 0 – Apps processor
- 1 – Modem processor
- 2 – LPASS processor

### 5.3.4 NPA log

NPA logs stands for Node Power Architecture logs and is a framework for managing dynamic resources with goals of system transparency and interface consistency.

Opening the log in an editor will show something like the following log snippet:

```
0x1DF2DCC: npa_issue_required_request (handle: 0xC07CA20C) (client:
"pwr") (request: 1) (resource: "/core/cpu/vdd")
0x1DF2DCC: request complete (handle: 0xC07CA20C) (sequence:
0x00016501) (request state:1) (active state:1)
```

```

1      0x1DFB7A1: npa_issue_required_request (handle: 0xC07CA20C) (client:
2      "pwr") (request: 1) (resource: "/core/cpu/vdd")
3      0x1DFB7A1:      request complete (handle: 0xC07CA20C) (sequence:
4      0x00016501) (request state:1) (active state:1)
5

```

The first column is the timestamp. This is in sclk ticks, although that is dependent on a system that has access to the sclk counter.

The second entry is the operation that is being run. In the snippet above there are two pairs for the NPA Log messages, as NPA logs happen in pairs. They are the entry and exit from the framework. Above we have npa\_request\_resource and request complete. This indicates that a client is being used to request from a resource, and that the request has been completed and the new state applied.

Although these happen in pairs, they do not necessarily happen immediately after each other in the log. Each line has a parameter handle. These parameters match between the request and the request complete. In the example above, the handle is 0xC07CA20C.

As implied by the previous paragraph, after the operation field, each log message has a set of parameters. These vary by the operation, but they all follow the form:

(parameter name: parameter value).

Parsing the npa\_request\_resource from above, the client making the request is "pwr". That client is requesting 1 from the resource "/core/cpu/vdd". The units of the resource are not shown in the log with each request. Units are defined individually for each resource.

Parsing the request complete from above, the clients (request state) are requesting 480000 KHz from the resource, but the actual state of the resource (active state) is 384000 KHz. The difference between the requested and actual values is due to a decision internal to the /clk/cpu resource.

The final piece of information in the request complete above is the sequence number. There are two things that can be gleaned from this number.

- Its primary function is to indicate the ordering of operations. This is mainly for automated tool parsing.
- The second piece of information is the low bit of the value. When this bit is a 1 (as in our case), the request is redundant (the client is not changing its requested value), so the request is not being propagated through the system.

Here is the list of some of the additional messages you might encounter in a log. This is not a complete list.

- npa\_alias\_resource – A resource is getting an additional name (an alias)
- npa\_define\_node – A node (collection of resources) is being defined
- npa\_create\_sync\_client – A synchronous client of a resource is being created
- npa\_create\_async\_client – An asynchronous client of a resource is being created
- npa\_create\_change\_event – A client has requested to be notified when a resource changes value
- npa\_cancel\_request – A client is done with the resource, and is removing its previous request

## 5.4 Voice troubleshooting

Follow the procedures in Section 2.3, Figure 2-2 of [Q5] to enable PCM logging points in the voice path using QXDM. Then follow Section 2.5.3 to parse the PCM logs using QCAT.

### 5.4.1 Voice call is not working

Usually, the first step is getting all the right logs and checking if the call is set up correctly. This includes ensuring that device and stream sessions are created, attached, and then moved to run. Other things such as media type, calibration and timing commands also need to happen, but these are the most necessary ones.

If all this is correct, check for data path messages. You should see one message from each of the Tx and Rx voice matrices every 20 ms indicating that they delivered a buffer downstream. If one or the other is missing, it indicates the corresponding path is not running correctly.

If that is also fine, the next step is to check the pcm logs. There should be valid data at all points, i.e., vptx input/output, venc/vdec input/output, and vprx output. If any of these is missing/silence, you can start looking at why that might happen, e.g., if Tx is not working, it is possible AFE is not running.

### 5.4.2 Voice is garbled during voice call

For an uplink path, start from checking at AFE Tx pcm and then vptx output pcm. Most garbling tends to be introduced either during encoding or from the EC. This can narrow down where it is introduced.

For a downlink path, check if garbling is present in Rx pkt and in vdec stream input to determine which might be the problem.

### 5.4.3 Voice choppiness during voice call

Choppiness is almost always due to timing issues or very rarely due to MIPS issues. Check for uplink timing by seeing if the encoder missed packets. The encoder prints messages when it has nothing to deliver. On the MSM8974, things are a little different due to the presence of VDS (Voice Delivery Service). If you see consistent packet misses from the encoder, it usually means that a bug is in vptx or the clocks are set too low.

For downlink choppiness, the decoder prints messages when it does not have a packet available and it has to do erasures. Seeing a lot of packet misses generally means bad downlink timing.

Both these issues can be analyzed by timing the APR events for encode, decode request, and decode packet.

## 5.5 Audio troubleshooting

Follow Section 2.2 Figure 2-1 of [Q5] to enable PCM logging points in the voice path using QXDM, then follow Section 2.5.3 to parse the PCM logs using QCAT. The log points contain both playback and record paths point taping at each major block in both paths. There could be many reasons for Audio not working as expected, this document deals with only the LPASS side of it, assuming everything else from Apps or other clients is as expected.

Thus the sequence of commands is assumed to be following as required in ISOD documents.

## 5.5.1 Audio playback

In this section, debug techniques specific to audio playback are discussed

### 5.5.1.1 Audio is not working

#### ASM side

Check to see if the buffers are being exchanged between the ADSP and apps processor. If they are being exchanged, then check the logs at the different PCM logging points to determine which module is causing the mute. If they are not being exchanged, then check if the stream is correctly routed to the device in the matrix map routing command.

To check the buffer exchanges look for the ASM messages from apps to ADSP, 0x10BD9 (ASM\_DATA\_CMD\_WRITE) and from ADSP to Apps 0x10BDF (ASM\_DATA\_EVENT\_WRITE\_DONE), as shown in the example below. In a good case there should be an equal number of both messages.

```
06:41:02.506      85  ./src/AudioStreamMgr_AprIf.cpp  AudioStreamMgr:
Rec cmd 0x10bd9 at [Addr=0x407, Port=0x101]
06:41:02.506     299  ../datalink/tal_smd/src/aprtalsmd.c
apr_tal_smd_rx_cb: bytes_remaining 44, size 0
06:41:02.516     279  ./src/AudioStreamMgr_AprIf.cpp
AudioStreamMgr:Port=0x101: ISR status 0x0, ASM_DATA_EVENT_WRITE_DONE 0x
10bdf , [0x0,0]
```

The Elite framework can queue up few of the WRITE messages and buffers before decoding and producing WRITE\_DONE messages.

One of the reasons the buffer exchange will not occur is due to ADSP stall or Apps stall. To check from the LPASS side, check PCM log points at other points as well, along with merged trace buffers. If Apps is stalled, then after the WRITE\_DONE event, Apps may not provide any further WRITE events, but the session is still active and ADSP expects WRITE events.

Sometimes Apps may send the buffers for LPASS to consume, but LPASS may not respond even after a long time

#### ADM side

Check the log-points and PCM data at those points.

### 5.5.1.2 Audio is garbled

Check the log-points to see which module is causing the distortion. It could be some calibration issue or it could be an issue with the media type set in the decoder. If it is a calibration issue, check with the Tuning team; if it is media type, check sampling rate, number of channels and other information being sent from the apps processor.

### 5.5.1.3 Audio is noisy or choppy

For a noisy case, see Section 5.5.1.2. If audio is choppy, check the timestamps of the buffers being delivered to the DSP. The DSP may be starved of data. You may be seeing a gap more than 10 ms in the timestamps between DATA\_WRITE events.

### 5.5.1.4 Audio clip does not end

Check for the EOS command

```
06:41:32.425      720  ./src/AudioDecSvc.cpp  Decoder svc received EOS
cmd!!!!
06:41:32.425     1407  ./src/AudioDecSvc.cpp  No data in OP buffer.
return it to Q!
06:41:32.425      857  ./src/AudioDecSvc.cpp  DecSvc: sending EOS
downstream
06:41:32.430      647  ./src/MixerSvc_OutPortHandler.cpp  MtMx #0 o/p
port 0: Sending EOS from i/p port 0 DS. EOSFormat: 1
06:41:32.450     2730  ./src/AFEPortManager.cpp  EoS msg
port_id=4000,client_id=0, EOSFormat:1
```

If this command is reached but still audio is playing, this message may not have reached all the downstream services. Check to see if the EOS has reached to Matrix and AFE.

If the EOS command is not seen in the logs, check from the Apps side.

### 5.5.1.5 Audio is not in sync with video

Check that all buffers have the time stamp validity flag or time stamp continuation flag set. Even if one buffer does not have this set, the DSP will go to nontimestamp honoring mode.

### 5.5.1.6 Audio quality is not as expected for AAC+/eAAC+ clips

See Section 5.5.1.2.

### 5.5.1.7 Audio playback is too fast or too slow

The sampling rate may be set wrong; check the logs and Apps side setting for media format.

## 5.5.2 Audio pre/postprocessing

This section describes how to debug the pre/post-processing features in audio playback and record use cases.

### 5.5.2.1 Specific pre/postprocessing feature is not working

Check that the set params went through correctly. Also check that the feature supports the media type of the data being passed, e.g., some modules do not support 24 bit audio.

```
06:41:32.426      344  ./src/appi_downmix.cpp  APPI Downmix Set param
done
06:41:32.426      700  ../core/src/aprv2_drv.c  free(00000407,C07493A0)
06:41:32.426      442  ./src/appi_genericresampler24src.cpp  APPI
Resampler reset
06:41:32.426      494  ./src/appi_genericresampler24src.cpp  APPI
Resampler Set Param done
06:41:32.426      528  ./src/appi_softvolumecontrols.cpp  APPI
SoftVolumeControls Set param done
06:41:32.426      657  ./src/appi_combopp.cpp  APPI ComboPP Eq, QCPR
reset
06:41:32.426      674  ./src/appi_combopp.cpp  APPI ComboPP Set param
done
06:41:32.426      444  ./src/appi_spa.cpp  APPI SPA Set param done
06:41:32.426      429  ./src/appi_peakmeter.c  APPI Example Buf Reset
```

### 5.5.2.2 Audio is choppy when pre/post-processing features are enabled

The DSP may be running out of MIPS. Check if the use case is supported.

## 5.5.3 Audio recording

This section discusses debug techniques for audio recording.

### 5.5.3.1 Audio is not recording

This is the same as Section 5.5.1.1, but instead of WRITE events, check for ASM\_DATA\_CMD\_READ (0x00010BDA) and ASM\_DATA\_EVENT\_READ\_DONE (0x00010BE0) messages

### 5.5.3.2 Recorded file playback is choppy/encoded frames might be missing

See Section 5.5.1.3.

### 5.5.3.3 Audio recording is noisy

See Section 5.5.1.2.

1

2

## 3

4

5



6

1



## 6.2 Finding the physical→virtual address mapping for ADSP image sections

To find the physical→virtual address mapping for ADSP image sections, enter:

```
qdsp6-objdump -h <ADSP image>
```

An example of the output for this command is shown here.

```
http://cygwin.com/cygwin-ug-net/using.html#using-pathnames
dsp1.mbn:      file format elf32-littleqdsp6

Sections:
Idx Name          Size      VMA           LMA           File off  Algn
 0 .start          000000c30  c00000000  467000000  0001f000  2**5
   CONTENTS, ALLOC, LOAD, CODE
 1 .init           000000060  c0000c40  46700c40  0001fc40  2**5
   CONTENTS, ALLOC, LOAD, READONLY, CODE
 2 .text           00313c50  c0001000  46701000  00020000  2**12
   CONTENTS, ALLOC, LOAD, READONLY, CODE
 3 .fini           000000040  c0314c60  46a14c60  00333c60  2**5
   CONTENTS, ALLOC, LOAD, READONLY, CODE
 4 .rodata         0014f6f0  c0315000  46a15000  00334000  2**5
   CONTENTS, ALLOC, LOAD, READONLY, DATA
 5 .data           000adc20  c0466000  46b66000  00484000  2**12
   CONTENTS, ALLOC, LOAD, DATA
 6 .eh_frame       00001074  c0513c20  46c13c20  00531c20  2**5
   CONTENTS, ALLOC, LOAD, DATA
 7 .gcc_except_table 00000718  c0514c94  46c14c94  00532c94  2**2
   CONTENTS, ALLOC, LOAD, DATA
 8 .ctors          00000020  c05153ac  46c153ac  005333ac  2**2
   CONTENTS, ALLOC, LOAD, DATA
 9 .dtors          0000000c  c05153cc  46c153cc  005333cc  2**2
   CONTENTS, ALLOC, LOAD, DATA
10 .sdata          0000e898  c0580000  46c80000  00534000  2**3
```

As shown, the .text section starts at virtual address 0xc0001000, which corresponds to physical address 0x46701000.